Bölüm 2: Oyun Kuramına Giriş

Algoritmik Oyun Kuramı

Normal ve genişletilmiş form | Oyunların sınıflandırılması | En iyi tepki | Karışık denge | Klasik 2×2 oyunlar | Kırkayak oyunu | Alt oyun mükemmel denge

Sercan KÜLCÜ · Giresun Üniversitesi · Bilgisayar Mühendisliği

TEMEL KAVRAMLAR

Öğrenme Çıktıları

Temsil

Bir etkileşimi normal form (matris) ya da genişletilmiş form (oyun ağacı) ile yazmak.

Sınıflandırma

Oyunları zamanlama, işbirliği, toplam kazanç, simetri, bilgi ve ufuk boyutlarında ayırt etmek.

Denge bulma

2×2 oyunlarda saf dengeleri en iyi tepki işaretlemesiyle, karışık dengeyi kayıtsızlık denklemiyle bulmak.

Geriye doğru çıkarım

Sıralı oyunları yapraktan köke çözmek ve alt oyun mükemmel dengeyi yorumlamak.

TEMEL KAVRAMLAR

Oyun Kuramı (Game Theory) Nedir?

- Rasyonel karar vericiler arasındaki **çatışma ve işbirliğini** inceleyen matematiksel modeller bütünü.
- Bir oyun; **oyuncular**, kullanabilecekleri **stratejiler** ve sonuçlara bağlı **kazançlarla** (payoff) tanımlanır.
- Her oyuncunun sonucu yalnızca kendi seçimine değil, **diğer oyuncuların seçimlerine** de bağlıdır: stratejik etkileşim.

Ekonomi
Fiyat rekabeti, pazarlık, açık artırma.

Bilgisayar bilimi
Ağ yönlendirme, çevrimiçi platformlar, çoklu ajan sistemleri.

Yapay zekâ
Oyun oynayan programlar, öğrenen ajanlar.

Algoritmik oyun kuramı iki soruyu birleştirir: **Ajanlar nasıl davranır?** ve **Dengeyi / iyi bir sonucu verimli hesaplayabilir miyiz?**

Algoritmik Oyun Kuramının Dört Sorusu

1. Hesaplama: denge bulunabilir mi?

Polinom zamanda çözülen problemler P sınıfındadır. Sıfır toplamlı oyunlar doğrusal programlamayla bu sınıftadır; genel oyunlarda Nash dengesi bulmak **PPAD-tam**, genelleştirilmiş n×n Hex ise **PSPACE-tam**dır.

2. Verimlilik: bencillik ne kaybettirir?

Anarşinin Bedeli = en kötü dengenin maliyeti / en iyi sonucun maliyeti. Mahkûmun İkileminde denge toplam 4 yıl, (Sessiz, Sessiz) 2 yıl: oran 4/2 = 2.

3. Tasarım: hangi kurallar?

Mekanizma tasarımı, oyuncuların özel bilgisini dürüstçe bildirmesini sağlayan (teşvik uyumlu) kuralları arar: açık artırmalar, eşleştirme, oylama.

4. Dinamik: oyuncular dengeye nasıl ulaşır?

Tekrarlanan ve çevrimiçi oyunlarda bilgi zamanla gelir; oyuncular geçmişten öğrenerek stratejisini uyarlar.

Bu bölüm, bu soruların hepsinin üzerine kurulduğu dili verir: oyunun temsili, sınıflandırılması ve küçük oyunlarda dengenin elle bulunması.

TEMEL KAVRAMLAR

Bir Oyunu Tanımlamak: Üç Soru

1. Oyuncular

Karar vericiler kimlerdir?

- İnsanlar mı?
- Hükümetler mi?
- Şirketler ya da bir şirketin çalışanı mı?
- Yazılım ajanları mı?

2. Eylemler / stratejiler

Oyuncular ne yapabilir?

- Açık artırmada teklif vermek
- Grevi bitirip bitirmemek
- Hisseyi ne zaman satmak
- Nasıl oy vermek

3. Kazançlar (payoff)

Oyuncuları ne motive eder?

- Kâr mı?
- Zaman, ceza, puan mı?
- Diğer oyuncuların durumu mu?

Kazançlar **fayda fonksiyonu** (utility function) $u_i(s_1, \ldots, s_n)$ ile verilir: oyuncu i'nin, herkesin seçtiği stratejilere bağlı değerlendirmesi. Kural: **büyük sayı = daha iyi**.

Normal Form (Stratejik Form)

- Oyuncuların eylemleri sonucunda **kazançlarını listeleyen** gösterim.
- Eylemlerin sırası ve zamanlaması dikkate alınmaz: oyuncular **aynı anda** (ya da birbirini görmeden) seçer.
- İki oyunculu oyunda **matris**: oyuncu 1 satırı, oyuncu 2 sütunu seçer.
- Hücrede **(u_1, u_2)**: önce satır, sonra sütun oyuncusunun kazancı.

satır ╲ sütun	Sessiz	İhanet
Sessiz	(−1, −1)	(−3, 0)
İhanet	(0, −3)	(−2, −2)

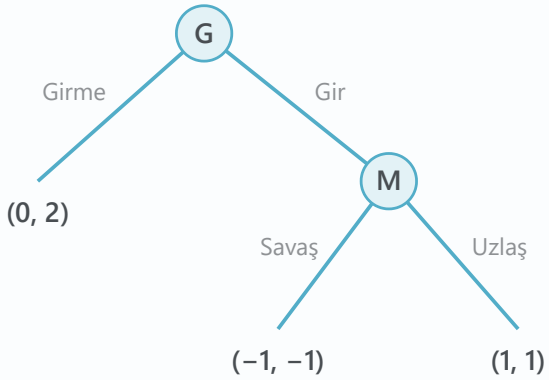
Örnek: Mahkûmun İkilemi (kazanç = −hapis yılı).

(Sessiz, İhanet) hücresi: oyuncu 1 → 3 yıl, oyuncu 2 → serbest.

TEMEL KAVRAMLAR

Genişletilmiş Form (Oyun Ağacı)

- Hareketlerin **zamanlamasını** ve sırasını dikkate alır.
- **Ağaç** yapısıyla temsil edilir:
  - **karar düğümü**: sırası gelen oyuncu
  - **dal**: bir eylem
  - **yaprak**: oyunun sonu ve kazançlar
- Örnek: **piyasaya giriş**. Giren firma (G) girip girmemeye karar verir; girerse mevcut firma (M) fiyat savaşı açar ya da uzlaşır.



Yapraklarda (G'nin kazancı, M'nin kazancı). Bu oyunu bölümün sonunda geriye doğru çıkarımla çözeceğiz.

Hatırlatma: Mahkûmun İkilemi

satır ↓	sütun oyuncusu: Sessiz	sütun oyuncusu: İhanet
Sessiz	(−1, −1)	(−3, **0**)
İhanet	(**0**, −3)	(**−2**, **−2**)

- **İhanet** her iki oyuncu için **baskın strateji**: karşı taraf ne yaparsa yapsın daha iyi ($0 > -1$ ve $-2 > -3$).
- Tek Nash dengesi (İhanet, İhanet): ikisi de 2 yıl yatar. Oysa ikisi de sessiz kalsa (işbirliği) 1'er yıl yatarlardı.
- Ders: **bireysel rasyonellik** herkes için daha kötü bir ortak sonuca götürebilir; denge **Pareto optimal değildir**.

Turuncu kutular en iyi tepkileri, çerçeve dengeyi gösterir.

Kod İzleme: `eleme(A, Bt, S, T)`

	sütun oyuncusu 0	1	2
0	4 , 3	5 , 1	6 , 2
1	2 , 1	8 , 4	3 , 6
2	3 , 0	9 , 6	2 , 8

Rakibin **her** seçimine karşı kesin daha iyisi olan strateji kesin baskılanır, elenir.

Adım 1 / 8

```
1  def eleme(A, Bt, S, T):
▶      degisti = True
3      while degisti:
4          degisti = False
5          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
6              for s in X:
7                  if any(all(K[r][t] > K[s][t] for t in Y)
8                         for r in X if r != s):
9                      X.remove(s); print(ad, s)
10                     degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
▶  print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

DEĞİŞKENLER

`S = [0, 1, 2]` `T = [0, 1, 2]` `degisti = True`

ÇIKTI

Kod İzleme: `eleme(A, Bt, S, T)`

	sütun oyuncusu 0	1	2
0	4 , 3	5 , 1	6 , 2
1	2 , 1	8 , 4	3 , 6
2	3 , 0	9 , 6	2 , 8

1. tur, satırlar: hiçbiri baskılanmıyor; ör. satır 1, sütun 1'de satır 0'dan iyi (8 > 5).

Adım 2 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
▶      while degisti:
▶          degisti = False
▶          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
▶              for s in X:
▶                  if any(all(K[r][t] > K[s][t] for t in Y)
▶                         for r in X if r != s):
9                      X.remove(s); print(ad, s)
10                     degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

DEĞİŞKENLER

`ad = satır` `S = [0, 1, 2]` `T = [0, 1, 2]` `s = 2` `degisti = False`

ÇIKTI

Kod İzleme: `eleme(A, Bt, S, T)`

	sütun oyuncusu 0	1	2
0	4 , 3	5 , 1	6 , 2
1	2 , 1	8 , 4	3 , 6
2	3 , 0	9 , 6	2 , 8

Sütunlar: sütun 1'i sütun 2 kesin yener: 2 > 1, 6 > 4, 8 > 6. Sütun 1 **elenir**.

Adım 3 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
3      while degisti:
4          degisti = False
▶          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
▶              for s in X:
▶                  if any(all(K[r][t] > K[s][t] for t in Y)
▶                         for r in X if r != s):
▶                      X.remove(s); print(ad, s)
▶                      degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

DEĞİŞKENLER

`ad = sütun` `S = [0, 1, 2]` `T = [0, 2]` `s = 1` `degisti = True`

ÇIKTI

```
sütun 1
```

Kod İzleme: `eleme(A, Bt, S, T)`



2. tur: sütun 1 gidince satır 0, satır 1'i kesin yener (4 > 2, 6 > 3): **elenir**.

Adım 4 / 8

```
 1 def eleme(A, Bt, S, T):
 2     degisti = True
 ▶     while degisti:
 ▶         degisti = False
 ▶         for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
 ▶             for s in X:
 ▶                 if any(all(K[r][t] > K[s][t] for t in Y)
 ▶                        for r in X if r != s):
 ▶                     X.remove(s); print(ad, s)
 ▶                     degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

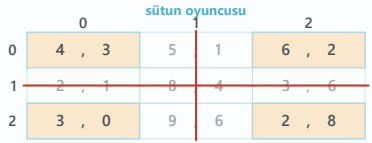
DEĞİŞKENLER

`ad = satır` `S = [0, 2]` `T = [0, 2]` `s = 1` `degisti = True`

ÇIKTI

```
sütun 1
satır 1
```

Kod İzleme: `eleme(A, Bt, S, T)`



Sütun 0 satır 0'da (3 > 2), sütun 2 satır 2'de (8 > 0) iyi: eleme yok.

Adım 5 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
3      while degisti:
4          degisti = False
▶          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
▶              for s in X:
▶                  if any(all(K[r][t] > K[s][t] for t in Y)
▶                         for r in X if r != s):
9                      X.remove(s); print(ad, s)
10                     degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

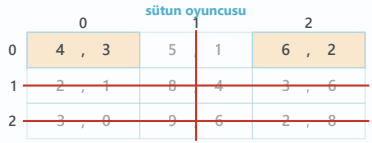
DEĞİŞKENLER

`ad = sütun` `S = [0, 2]` `T = [0, 2]` `s = 2` `degisti = True`

ÇIKTI

```
sütun 1
satır 1
```

Kod İzleme: `eleme(A, Bt, S, T)`



3. tur: satır 0, satır 2'yi kesin yener: 4 > 3, 6 > 2. Satır 2 **elenir**.

Adım 6 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
▶      while degisti:
▶          degisti = False
▶          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
▶              for s in X:
▶                  if any(all(K[r][t] > K[s][t] for t in Y)
▶                         for r in X if r != s):
▶                      X.remove(s); print(ad, s)
▶                      degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

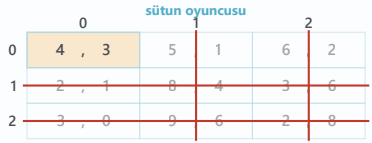
DEĞİŞKENLER

`ad = satır` `S = [0]` `T = [0, 2]` `s = 2` `degisti = True`

ÇIKTI

```
sütun 1
satır 1
satır 2
```

Kod İzleme: `eleme(A, Bt, S, T)`



Yalnız satır 0 kaldı: orada sütun 0 (3), sütun 2'yi (2) yener. Sütun 2 **elenir**.

Adım 7 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
3      while degisti:
4          degisti = False
           for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
               for s in X:
                   if any(all(K[r][t] > K[s][t] for t in Y)
                          for r in X if r != s):
                       X.remove(s); print(ad, s)
                       degisti = True; break
11     return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
13 print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

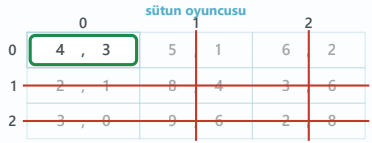
DEĞİŞKENLER

`ad = sütun` `S = [0]` `T = [0]` `s = 2` `degisti = True`

ÇIKTI

```
sütun 1
satır 1
satır 2
sütun 2
```

Kod İzleme: `eleme(A, Bt, S, T)`



Değişiklik yok: kalan (0, 0), kazanç (4, 3). Eleme sırası bu sonucu değiştirmez.

Adım 8 / 8

```
1  def eleme(A, Bt, S, T):
2      degisti = True
▶      while degisti:
4          degisti = False
5          for K, X, Y, ad in [(A, S, T, 'satır'), (Bt, T, S, 'sütun')]:
6              for s in X:
7                  if any(all(K[r][t] > K[s][t] for t in Y)
8                         for r in X if r != s):
9                      X.remove(s); print(ad, s)
10                     degisti = True; break
▶      return S, T
12 A, Bt = [[4,5,6], [2,8,3], [3,9,2]], [[3,1,0], [1,4,6], [2,6,8]]
▶ print(eleme(A, Bt, [0, 1, 2], [0, 1, 2]))     # Bt: sütun, devrik
```

DEĞİŞKENLER

`S = [0]` `T = [0]`

`degisti = False`

ÇIKTI

```
sütun 1
satır 1
satır 2
sütun 2
([0], [0])
```

TEMEL KAVRAMLAR

Temel Çözüm Kavramları

En iyi tepki (best response)

Diğerlerinin stratejileri sabitken oyuncuya en yüksek kazancı veren strateji. Matriste: her sütunda satır oyuncusunun, her satırda sütun oyuncusunun en büyük kazancını işaretle.

Nash dengesi (Nash equilibrium)

Her oyuncunun stratejisi diğerlerine en iyi tepki olan strateji profili: kimse **tek başına** sapmakla kazanamaz. Matriste: iki kazancı da işaretli hücre.

Baskın strateji (dominant strategy)

Rakip ne yaparsa yapsın diğer stratejilerden daha iyi olan strateji. Varsa oyuncu onu oynar (Mahkûmun İkilemi'nde İhanet).

Pareto optimal

Bir sonuç, **hiç kimsenin durumu kötüleşmeden** en az bir oyuncunun durumu iyileştirilemiyorsa Pareto optimaldir. İyileştirme mümkünse sonuç Pareto'ya göre *baskılanmıştır*.

Kod İzleme: `saf_dengeler(A, B)`

	sütun oyuncusu 0	1	2
0	3 , 2	1 , 1	0 , 3
1	2 , 0	2 , 2	1 , 1
2	0 , 1	3 , 0	2 , 2

Her hücrede iki soru: satır bu sütuna, sütun bu satıra en iyi tepki mi? İkisi de evetse denge.

Adım 1 / 11

```
1  def saf_dengeler(A, B):
▶      m, n = len(A), len(A[0])
▶      NE = []
4      for i in range(m):
5          for j in range(n):
6              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
7              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
8              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
▶  A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
▶  B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
▶  print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`m = 3` `n = 3` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`

	sütun oyuncusu 0	1	2
0	3 , 2	1 , 1	0 , 3
1	2 , 0	2 , 2	1 , 1
2	0 , 1	3 , 0	2 , 2

(0, 0): sütun 0'da satırın kazançları 3, 2, 0 → 3 en büyük: **en iyi tepki**. Sütunun 2'si satır 0'da en büyük değil.

Adım 2 / 11

```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 0` `j = 0` `satir_iyi = True` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(0, 1): sütun 1'de satırın en büyüğü 3 (satır 2), burada 1. Sütunun 1'i de en büyük değil.

Adım 3 / 11

```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 0` `j = 1` `satir_iyi = False` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`

	sütun oyuncusu 0	1	2
0	**3**, 2	1 , 1	0 , **3**
1	2 , 0	2 , 2	1 , 1
2	0 , 1	3 , 0	2 , 2

(0, 2): satır 0'da sütunun en büyüğü 3 burada → sütun için en iyi tepki. Satırın 0'ı ise sütun 2'de en küçük.

Adım 4 / 11

```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 0` `j = 2` `satir_iyi = False` `sutun_iyi = True` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(1, 0): 2 < 3 ve 0 < 2: iki oyuncu da tek başına sapmak ister.

Adım 5 / 11

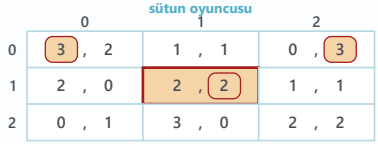
```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 1` `j = 0` `satir_iyi = False` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(1, 1): satır 1'de sütunun en büyüğü 2 burada (işaret). Satırın 2'si sütun 1'de 3'ün altında.

Adım 6 / 11

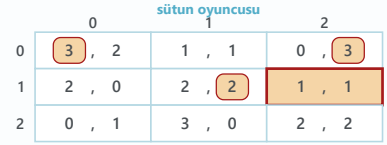
```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 1` `j = 1` `satir_iyi = False` `sutun_iyi = True` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(1, 2): 1 < 2 (satır) ve 1 < 2 (sütun): işaret yok.


```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 1` `j = 2` `satir_iyi = False` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(2, 0): satırın 0'ı sütun 0'da en küçük; sütunun 1'i satır 2'de 2'nin altında.

Adım 8 / 11

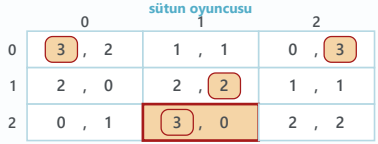
```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 2` `j = 0` `satir_iyi = False` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(2, 1): sütun 1'de satırın en büyüğü 3 burada (işaret); sütunun 0'ı ise en küçük.

Adım 9 / 11

```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 2` `j = 1` `satir_iyi = True` `sutun_iyi = False` `NE = []`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



(2, 2): satırın 2'si sütun 2'de, sütunun 2'si satır 2'de en büyük. **İki işaret** → Nash dengesi.

Adım 10 / 11

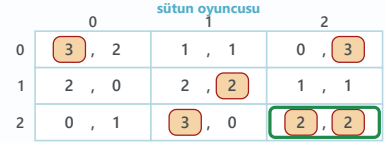
```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
▶      for i in range(m):
▶          for j in range(n):
▶              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
▶              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
▶              if satir_iyi and sutun_iyi:
▶                  NE.append((i, j))
10     return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
13 print(saf_dengeler(A, B))
```

DEĞİŞKENLER

`i = 2` `j = 2` `satir_iyi = True` `sutun_iyi = True` `NE = [(2, 2)]`

ÇIKTI

Kod İzleme: `saf_dengeler(A, B)`



Tek saf Nash dengesi (2, 2). Maliyet: m·n hücre, hücre başına m + n karşılaştırma → O(mn(m + n)).

Adım 11 / 11

```
1  def saf_dengeler(A, B):
2      m, n = len(A), len(A[0])
3      NE = []
4      for i in range(m):
5          for j in range(n):
6              satir_iyi = A[i][j] == max(A[k][j] for k in
   range(m))
7              sutun_iyi = B[i][j] == max(B[i][k] for k in
   range(n))
8              if satir_iyi and sutun_iyi:
9                  NE.append((i, j))
▶      return NE
11 A = [[3, 1, 0], [2, 2, 1], [0, 3, 2]]
12 B = [[2, 1, 3], [0, 2, 1], [1, 0, 2]]
▶ print(saf_dengeler(A, B))
```

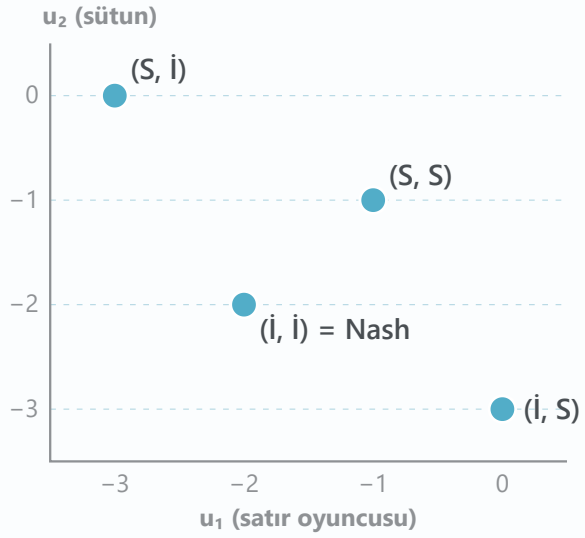
DEĞİŞKENLER

`NE = [(2, 2)]`

ÇIKTI

```
[(2, 2)]
```

Pareto Optimallik: Kazanç Uzayında Bakış



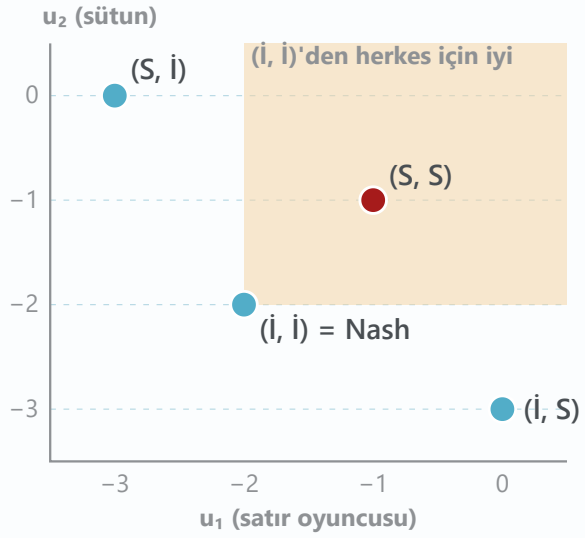
Her sonuç bir nokta: (u_1, u_2). Mahkûmun İkilemi'nin dört sonucu (S: Sessiz, İ: İhanet), kazanç = −(hapis yılı).

Adım 1 / 3

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Pareto Optimallik: Kazanç Uzayında Bakış

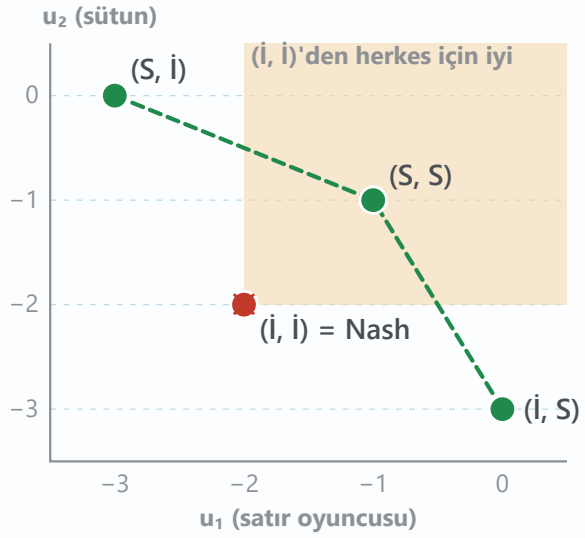


Nash dengesi (İ, İ) = (−2, −2). Turuncu bölgede ikisine de daha iyi bir sonuç var: **(S, S)**.

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Pareto Optimallik: Kazanç Uzayında Bakış



Pareto optimal: (S, S), (S, İ), (İ, S) (yeşil). Tek Nash dengesi olan (İ, İ) ise Pareto baskılanır.

- **Pareto baskınlık:** x, y'yi baskılar ⇔ herkes için u(x) ≥ u(y) ve en az biri için >.
- **Pareto optimal:** hiçbir sonuç onu baskılamaz; kimseyi kötüleştirmeden kimse iyileştirilemez.
- Grafikte: noktanın sağ üst çeyreğinde başka sonuç yoksa Pareto optimaldir.
- Pareto optimallik **adalet değildir**: (S, İ) de Pareto optimaldir ama biri 3 yıl yatar.

Denge (kararlılık) ile verimlilik ayrı sorulardır; aradaki kayıp Anarşinin Bedeli ile ölçülür.

Karışık Strateji ve Kayıtsızlık İlkesi

- **Saf strateji** (pure): bir eylemi kesin seçmek. **Karışık strateji** (mixed): eylemler üzerinde olasılık dağılımı, ör. %60 Opera, %40 Futbol.
- Karışık oynayan oyuncu, karıştırdığı eylemler arasında **kayıtsız** olmalıdır; biri daha iyi olsaydı onu kesin seçerdi.
- Bu yüzden her oyuncu, **rakibini kayıtsız bırakan** olasılığı seçer.
- Her sonlu oyunda en az bir (saf ya da karışık) Nash dengesi vardır (Nash, 1950).

2×2 oyun için tarif

1. q = P(sütun oyuncusu 1. sütunu oynar)
2. Satır oyuncusunun iki satırdaki beklenen kazancını q cinsinden yaz: E_1[satır 1], E_1[satır 2].
3. Eşitle ve q*'ı çöz.
4. Aynısını p = P(satır 1) için sütun oyuncusunun kazançlarıyla yap.
5. 0 < p*, q* < 1 ise karışık denge budur.

Oyunların Sınıflandırılması

Boyut	Bir uç	Diğer uç	Sorulan soru
Zamanlama	Eşzamanlı	Sıralı	Rakibin hamlesini görerek mi oynuyorum?
Anlaşma	İşbirlikçi	İşbirlikçi olmayan	Bağlayıcı anlaşma yapılabilir mi?
Toplam kazanç	Sıfır toplamlı	Sıfır toplamlı olmayan	Birinin kazancı diğerinin kaybı mı?
Simetri	Simetrik	Asimetrik	Oyuncuların yerini değiştirince oyun aynı mı?
Bilgi	Mükemmel bilgi	Mükemmel olmayan bilgi	Geçmiş hamlelerin hepsini görüyor muyum?
Ufuk	Sonlu	Sonsuz	Oyunun belirli bir sonu var mı?

Boyutlar birbirinden bağımsızdır: satranç sıralı, işbirlikçi olmayan, sıfır toplamlı, mükemmel bilgili ve sonlu bir oyundur.

Eşzamanlı ve Sıralı Oyunlar

Eşzamanlı oyunlar

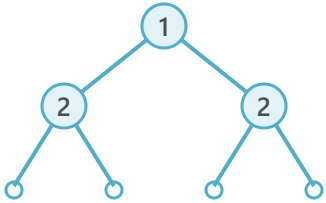
	A	B
A	?	?
B	?	?

normal form: herkes birlikte seçer

- Oyuncular aynı anda ya da birbirinin hamlesinden habersiz seçer.
- Genellikle normal formla gösterilir; stratejik belirsizlik yüksektir.

`Mahkûmun İkilemi, Taş–Kağıt–Makas, kapalı zarf teklif`

Sıralı oyunlar

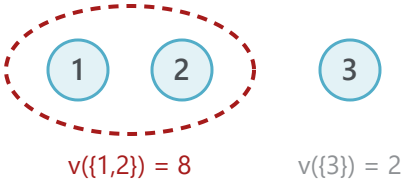


- Oyuncular farklı karar noktalarında hareket eder; sonraki oyuncu önceki hamleleri (tamamen ya da kısmen) görür.
- Genişletilmiş formla gösterilir; geriye doğru çıkarımla çözülebilir.

`Satranç, pazarlık, piyasaya giriş`

İşbirlikçi ve İşbirlikçi Olmayan Oyunlar

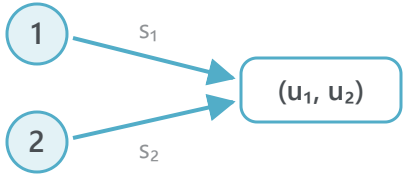
İşbirlikçi (cooperative) oyunlar



- Oyuncular dışarıdan uygulanan **bağlayıcı anlaşmalar** yapabilir (ör. sözleşme hukuku), koalisyon kurar.
- Soru: koalisyon ne üretir, değer nasıl paylaşılır? (çekirdek, Shapley değeri)

`Ortak girişim, maliyet paylaşımı, birleşme`

İşbirlikçi olmayan (non-cooperative)



- **Bağlayıcı anlaşma yoktur**; her oyuncu kendi stratejisini kendi çıkarı için seçer.
- İşbirliği yine oluşabilir, ama anlaşma **kendi kendini uygulamalıdır** (dengede olmalı).

`Rekabetçi piyasalar, Mahkûmun İkilemi`

Sıfır Toplamlı ve Sıfır Toplamlı Olmayan Oyunlar

Sıfır toplamlı (zero-sum)

	Y	T
Y	1, −1	−1, 1
T	−1, 1	1, −1

her hücrede $u_1 + u_2 = 0$

- Bir oyuncunun kazancı diğerinin kaybıdır; toplam her sonuçta sabittir (sıfır).
- Saf rekabet: ortak çıkar yoktur; çözüm kavramı minimakstır.

`Satranç, Yazı–Tura Eşleştirme, penaltı atışı`

Sıfır toplamlı olmayan

	S	İ
S	−1, −1	−3, 0
İ	0, −3	−2, −2

toplam −2 ile −4 arasında değişir

- Toplam kazanç sonuca göre değişir; herkesin kazandığı (ya da kaybettiği) sonuçlar olabilir.
- İşbirliği ve rekabet bir aradadır; iletişim ve müzakere sonucu değiştirebilir.

`Ticaret müzakeresi, iklim anlaşmaları`

Simetrik ve Asimetrik Oyunlar

Simetrik

	G	T
G	2, 2	0, 1
T	1, 0	1, 1

$u_1(G,T) = u_2(T,G) = 0$

- Oyuncuların strateji kümeleri aynı ve **$u_1(a, b) = u_2(b, a)$**: kazanç yalnızca stratejiye bağlı, kimin oynadığına değil.
- Matris "köşegene göre aynalı" görünür.

`Mahkûmun İkilemi, Geyik Avı, Tavuk`

Asimetrik

	O	F
O	3, 2	0, 0
F	0, 0	2, 3

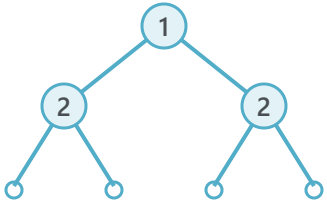
$u_1(O,O) = 3 \neq u_2(O,O) = 2$

- Oyuncuların rolleri, stratejileri ya da kazançları farklıdır.
- Aynı sonuç oyunculara farklı değer verir.

`Cinsiyetler Savaşı, piyasaya giriş, Yazı-Tura`

Mükemmel Bilgi ve Mükemmel Olmayan Bilgi

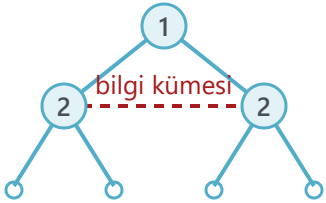
Mükemmel bilgi (perfect information)



- Karar anında o ana kadarki **tüm hamle geçmişi** bilinir.
- Geleceği bilmek demek değildir.

`Satranç, dama, go, Hex, Connect Four`

Mükemmel olmayan bilgi (imperfect)



- Bazı geçmiş hamleler görülmez; **bilgi kümesi** ayırt edilemeyen düğümleri birleştirir.
- Eşzamanlı oyunlar bu sınıftadır.

`Poker, kapalı zarf açık artırma`

Tam bilgi (complete) farklı bir kavramdır: oyuncuların tipleri, strateji kümeleri ve kazanç fonksiyonları herkesçe bilinir. Bu bilinmiyorsa **eksik bilgi** (incomplete) vardır → Bayesçi oyunlar.

Sonlu ve Sonsuz Ufuklu Oyunlar

Sonlu ufuk



- Oyun önceden belirli sayıda dönem ya da hamleden sonra biter.
- Son dönemin davranışı önceki kararları etkiler; **geriye doğru çıkarım** kullanılır.

`Tek seferlik pazarlık, T turlu tekrarlanan oyun`

Sonsuz ufuk



- Oyun belirsiz süre sürebilir; gelecek kazançlar **iskonto faktörü** $\delta \in (0, 1)$ ile tartılır: $\Sigma\, \delta^t u_t$.
- Gelecek yeterince önemliyse tehdit, ödül ve itibar işbirliğini destekler.

`Sürekli fiyat rekabeti, ortak kaynak kullanımı`

"Sonlu oyun" terimi ayrıca oyuncu ve strateji kümeleri sonlu olan stratejik oyunlar için de kullanılır (Nash teoremi bu anlamdadır).

Yazı-Tura Eşleştirme (Matching Pennies)

- İki oyuncu aynı anda bir madeni parayı **Yazı** ya da **Tura** yüzüyle gösterir.
- Paralar **eşleşirse** satır oyuncusu (Eşleştiren), **eşleşmezse** sütun oyuncusu (Ayrıştıran) 1 birim kazanır.
- Gerçek hayatta: penaltıda kaleci–atıcı, güvenlik denetiminde denetçi–kaçakçı.

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

Özellikler

- Sıfır toplamlı ($u_1 + u_2 = 0$), asimetrik.
- Çıkarlar tamamen zıt: saf dengesi yok.

ÇATIŞMA OYUNLARI

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , −1)	(−1 , 1)
Tura	(−1 , 1)	(1 , −1)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , −1)	(−1 , 1)
Tura	(−1 , 1)	(1 , −1)

Sütun oyuncusu **Yazı** oynarsa satır oyuncusu: Yazı → 1, Tura → −1 ⇒ en iyi tepki **Yazı**.

Adım 2 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
Yazı	(1 , −1)	(−1 , 1)
Tura	(−1 , 1)	(1 , −1)

Sütun oyuncusu **Tura** oynarsa satır oyuncusu: Yazı → −1, Tura → 1 ⇒ en iyi tepki **Tura**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
satır ↓ **Yazı**	(**1** , −1)	(−1 , **1**)
satır ↓ **Tura**	(−1 , 1)	(**1** , −1)

Satır oyuncusu **Yazı** oynarsa sütun oyuncusu: Yazı → −1, Tura → 1 ⇒ en iyi tepki **Tura**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
Yazı	(**1** , −1)	(−1 , **1**)
Tura	(−1 , **1**)	(**1** , −1)

Satır oyuncusu **Tura** oynarsa sütun oyuncusu: Yazı → 1, Tura → −1 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(**1** , −1)	(−1 , **1**)
Tura	(−1 , **1**)	(**1** , −1)

Hiçbir hücrede iki işaret yok ⇒ **saf Nash dengesi yok**. Her sonuçta oyunculardan biri sapmak ister (işaretler döngü oluşturur).

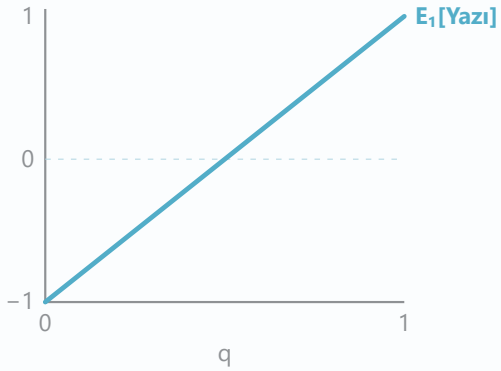
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + (-1) \cdot (1 - q) = 2q - 1$



Sütun oyuncusu Yazı'yı q olasılıkla oynasın. Satır oyuncusunun **Yazı** ile beklenen kazancı q'ya göre bir doğrudur.

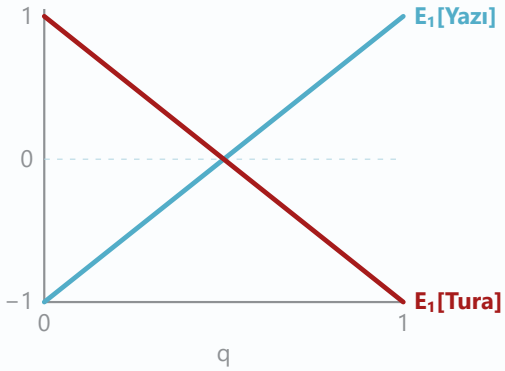
Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + (-1) \cdot (1 - q) = 2q - 1$

$E_1[\text{Tura}] = (-1) \cdot q + 1 \cdot (1 - q) = -2q + 1$



Tura için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

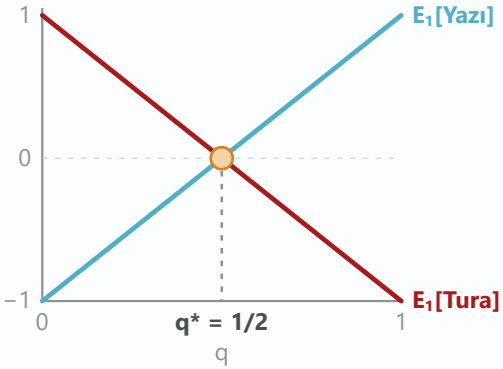
satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + (-1) \cdot (1 - q) = 2q - 1$

$E_1[\text{Tura}] = (-1) \cdot q + 1 \cdot (1 - q) = -2q + 1$

Kayıtsızlık: $2q - 1 = -2q + 1 \Rightarrow 4q = 2 \Rightarrow \mathbf{q^* = 1/2}$



Karışık oynamak ancak iki saf strateji **eşit** kazandırıyorsa mantıklıdır: kesişim q* = 1/2.

Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1\cdot q + (-1)\cdot(1-q) = 2q - 1$

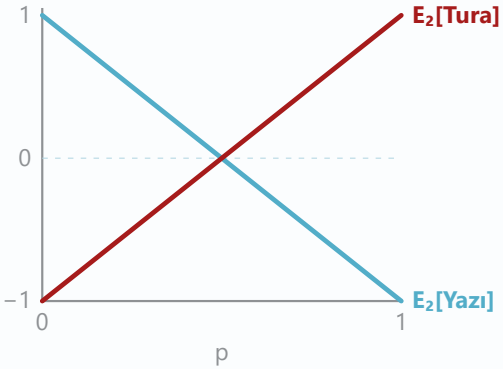
$E_1[\text{Tura}] = (-1)\cdot q + 1\cdot(1-q) = -2q + 1$

Kayıtsızlık: $2q - 1 = -2q + 1 \Rightarrow 4q = 2 \Rightarrow \mathbf{q^* = 1/2}$

p = P(satır oyuncusu **Yazı** oynar)

$E_2[\text{Yazı}] = (-1)\cdot p + 1\cdot(1-p) = -2p + 1$

$E_2[\text{Tura}] = 1\cdot p + (-1)\cdot(1-p) = 2p - 1$



Aynı mantık sütun oyuncusu için: satır oyuncusu Yazı'yı p olasılıkla oynasın; sütunun iki doğrusu.

Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1\cdot q + (-1)\cdot(1-q) = 2q - 1$

$E_1[\text{Tura}] = (-1)\cdot q + 1\cdot(1-q) = -2q + 1$

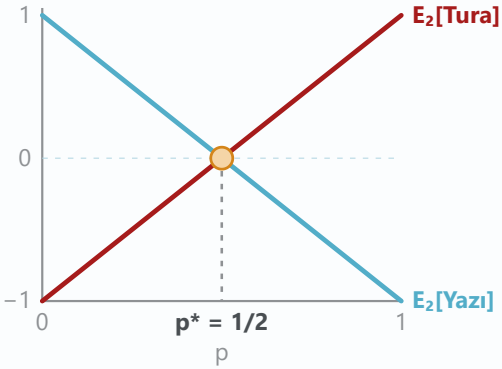
Kayıtsızlık: $2q - 1 = -2q + 1 \Rightarrow 4q = 2 \Rightarrow \mathbf{q^* = 1/2}$

p = P(satır oyuncusu **Yazı** oynar)

$E_2[\text{Yazı}] = (-1)\cdot p + 1\cdot(1-p) = -2p + 1$

$E_2[\text{Tura}] = 1\cdot p + (-1)\cdot(1-p) = 2p - 1$

Kayıtsızlık: $-2p + 1 = 2p - 1 \Rightarrow -4p = -2 \Rightarrow \mathbf{p^* = 1/2}$



Satır oyuncusu p* = 1/2 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Yazı-Tura Eşleştirme: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, −1)	(−1, 1)
Tura	(−1, 1)	(1, −1)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1\cdot q + (-1)\cdot(1-q) = 2q - 1$

$E_1[\text{Tura}] = (-1)\cdot q + 1\cdot(1-q) = -2q + 1$

Kayıtsızlık: $2q - 1 = -2q + 1 \Rightarrow 4q = 2 \Rightarrow \mathbf{q^* = 1/2}$

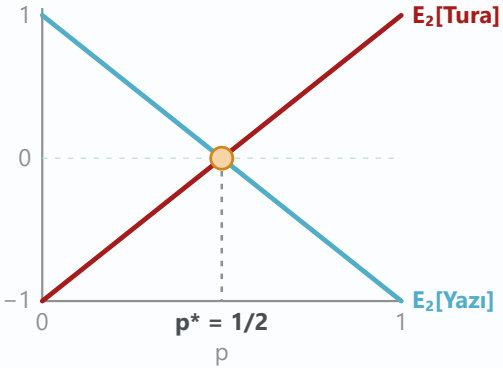
p = P(satır oyuncusu **Yazı** oynar)

$E_2[\text{Yazı}] = (-1)\cdot p + 1\cdot(1-p) = -2p + 1$

$E_2[\text{Tura}] = 1\cdot p + (-1)\cdot(1-p) = 2p - 1$

Kayıtsızlık: $-2p + 1 = 2p - 1 \Rightarrow -4p = -2 \Rightarrow \mathbf{p^* = 1/2}$

Denge: satır (Yazı 1/2, Tura 1/2), sütun (Yazı 1/2, Tura 1/2) · beklenen kazanç $u_1 = 0$, $u_2 = 0$



Sonuç: her oyuncu rakibini kayıtsız bırakan karışımı oynar; kimse tek başına sapmakla kazanamaz.

Yazı-Tura Eşleştirme: Sonuçların Yorumu

P(sonuç)	Yazı (1/2)	Tura (1/2)
Yazı (1/2)	1/4 = %25 (1, −1)	1/4 = %25 (−1, 1)
Tura (1/2)	1/4 = %25 (−1, 1)	1/4 = %25 (1, −1)

Karışık dengede dört sonuç eşit olasılıklı

Beklenen kazanç

$u_1 = 0$, $u_2 = 0$: oyun "adil"dir.

Eşleşme olasılığı

P(eşleşme) = 1/2 = %50: iki oyuncu da yarı yarıya kazanır.

Neden rastgele?

Önceden kestirilebilen her eğilim sömürülür: sütun %60 Yazı derse satır hep Yazı der ve kazanır.

Karışık strateji burada bir **gizlenme aracıdır**: amaç rakibe bilgi vermemek. Rastgelelik oyuncuyu sömürülemez kılar, ama kazanç garantisi vermez; ortalamada 0 elde edilir.

Tavuk (Chicken)

- İki sürücü dar bir yolda birbirine doğru ilerler. İlk **kıran** "tavuk" (korkak) olur (−1), düz giden kazanır (+1).
- İkisi de düz giderse **çarpışırlar** (−10, −10); ikisi de kırarsa berabere (0, 0).
- Kavşakta geçiş hakkı, fiyat savaşları, caydırma krizleri bu yapıdadır (Şahin–Güvercin oyunu da denktir).

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

Özellikler

- Simetrik, sıfır toplamlı değil.
- **Anti-koordinasyon**: farklı eylem seçmek istenir; bir **cesaret (caydırma) ikilemi**.

Tavuk: Saf Dengeleri İşaretle

	sütun oyuncusu: Kır	sütun oyuncusu: Düz git
satır ↓ **Kır**	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Sütun oyuncusu **Kır** oynarsa satır oyuncusu: Kır → 0, Düz git → 1 ⇒ en iyi tepki **Düz git**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Sütun oyuncusu **Düz git** oynarsa satır oyuncusu: Kır → −1, Düz git → −10 ⇒ en iyi tepki **Kır**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusu **Kır** oynarsa sütun oyuncusu: Kır → 0, Düz git → 1 ⇒ en iyi tepki **Düz git**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

Satır oyuncusu **Düz git** oynarsa sütun oyuncusu: Kır → −1, Düz git → −10 ⇒ en iyi tepki **Kır**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Kır	Düz git
Kır	(0 , 0)	(−1 , 1)
Düz git	(1 , −1)	(−10 , −10)

İki işaretli hücre(ler): **(Kır, Düz git), (Düz git, Kır)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

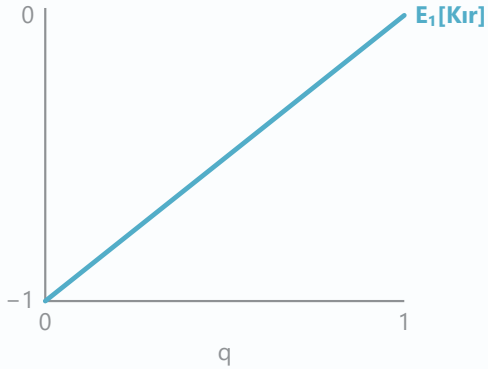
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Tavuk: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0 \cdot q + (-1) \cdot (1 - q) = q - 1$



Sütun oyuncusu Kır'ı q olasılıkla oynasın. Satır oyuncusunun **Kır** ile beklenen kazancı q'ya göre bir doğrudur.

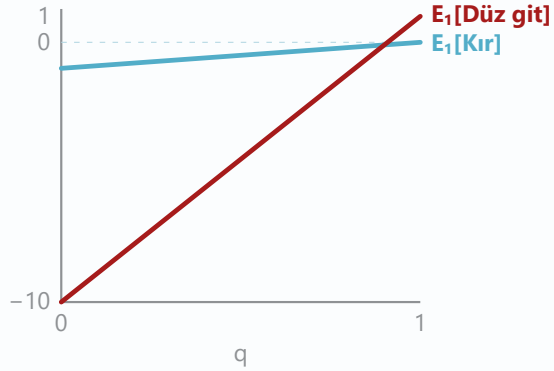
Tavuk: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0 \cdot q + (-1) \cdot (1 - q) = q - 1$

$E_1[\text{Düz git}] = 1 \cdot q + (-10) \cdot (1 - q) = 11q - 10$



Düz git için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Tavuk: Karışık Denge (Kayıtsızlık)

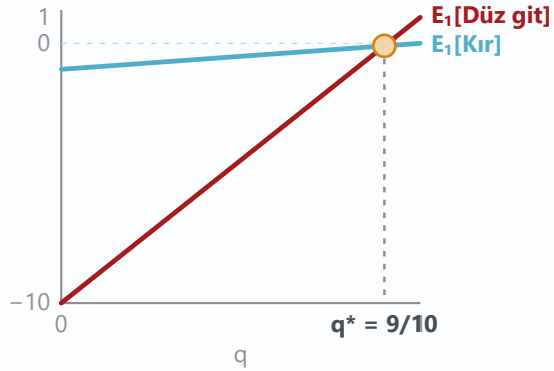
satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0\cdot q + (-1)\cdot(1-q) = q - 1$

$E_1[\text{Düz git}] = 1\cdot q + (-10)\cdot(1-q) = 11q - 10$

Kayıtsızlık: $q - 1 = 11q - 10 \Rightarrow -10q = -9 \Rightarrow \mathbf{q^* = 9/10}$



Karışık oynamak ancak iki saf strateji **eşit** kazandırıyorsa mantıklıdır: kesişim q* = 9/10.

Tavuk: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0\cdot q + (-1)\cdot(1-q) = q - 1$

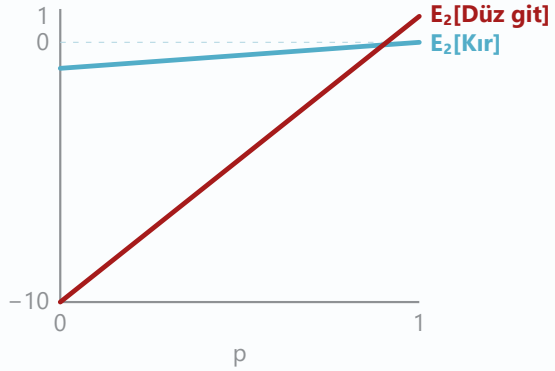
$E_1[\text{Düz git}] = 1\cdot q + (-10)\cdot(1-q) = 11q - 10$

Kayıtsızlık: $q - 1 = 11q - 10 \Rightarrow -10q = -9 \Rightarrow \mathbf{q^* = 9/10}$

p = P(satır oyuncusu **Kır** oynar)

$E_2[\text{Kır}] = 0\cdot p + (-1)\cdot(1-p) = p - 1$

$E_2[\text{Düz git}] = 1\cdot p + (-10)\cdot(1-p) = 11p - 10$



Aynı mantık sütun oyuncusu için: satır oyuncusu Kır'ı p olasılıkla oynasın; sütunun iki doğrusu.

Tavuk: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0\cdot q + (-1)\cdot(1-q) = q - 1$

$E_1[\text{Düz git}] = 1\cdot q + (-10)\cdot(1-q) = 11q - 10$

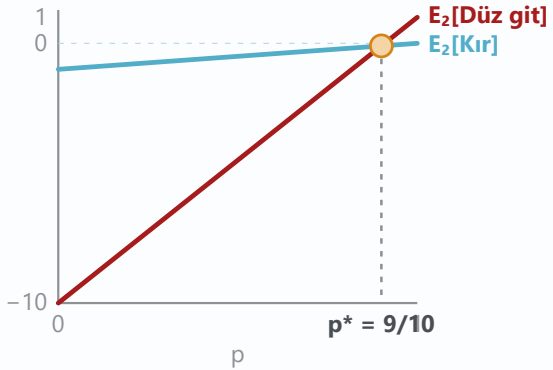
Kayıtsızlık: $q - 1 = 11q - 10 \Rightarrow -10q = -9 \Rightarrow \mathbf{q^* = 9/10}$

p = P(satır oyuncusu **Kır** oynar)

$E_2[\text{Kır}] = 0\cdot p + (-1)\cdot(1-p) = p - 1$

$E_2[\text{Düz git}] = 1\cdot p + (-10)\cdot(1-p) = 11p - 10$

Kayıtsızlık: $p - 1 = 11p - 10 \Rightarrow -10p = -9 \Rightarrow \mathbf{p^* = 9/10}$



Satır oyuncusu p* = 9/10 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Tavuk: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Kır	Düz git
Kır	(0, 0)	(−1, 1)
Düz git	(1, −1)	(−10, −10)

q = P(sütun oyuncusu **Kır** oynar)

$E_1[\text{Kır}] = 0\cdot q + (-1)\cdot(1-q) = q - 1$

$E_1[\text{Düz git}] = 1\cdot q + (-10)\cdot(1-q) = 11q - 10$

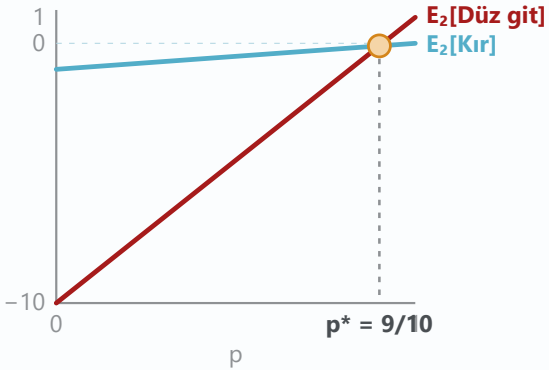
Kayıtsızlık: $q - 1 = 11q - 10 \Rightarrow -10q = -9 \Rightarrow \mathbf{q^* = 9/10}$

p = P(satır oyuncusu **Kır** oynar)

$E_2[\text{Kır}] = 0\cdot p + (-1)\cdot(1-p) = p - 1$
$E_2[\text{Düz git}] = 1\cdot p + (-10)\cdot(1-p) = 11p - 10$

Kayıtsızlık: $p - 1 = 11p - 10 \Rightarrow -10p = -9 \Rightarrow \mathbf{p^* = 9/10}$

Denge: satır (Kır 9/10, Düz git 1/10), sütun (Kır 9/10, Düz git 1/10) · beklenen kazanç $u_1 = -1/10$, $u_2 = -1/10$



Sonuç: her oyuncu rakibini kayıtsız bırakan karışımı oynar; kimse tek başına sapmakla kazanamaz.

Tavuk: Sonuçların Yorumu

P(sonuç)	Kır (9/10)	Düz git (1/10)
Kır (9/10)	81/100 = %81 (0, 0)	9/100 = %9 (−1, 1)
Düz git (1/10)	9/100 = %9 (1, −1)	1/100 = %1 (−10, −10)

Turuncu: çarpışma olasılığı 1/100

Beklenen kazanç

$u_1 = u_2 = -1/10$: saf dengelerde biri +1 alır, burada ikisi de eksidedir.

Çarpışma olasılığı

$P(\text{Düz, Düz}) = (1/10)^2 = 1/100$.

Trafik ışığı

1/2 olasılıkla "sen geç", 1/2 "ben geçerim" diyen ortak sinyal çarpışmayı sıfırlar; her sürücü 0 bekler.

Çarpışmanın bedeli büyüdükçe ($-10 \to -k$) kırma olasılığı artar: $p^* = (k - 1)/k$. Trafik ışığı bir **korelasyonlu denge** aracıdır: ışığa uymak herkesin çıkarınadır.

Cinsiyetler Savaşı (Battle of the Sexes)

- Bir çift akşam buluşacak; iletişim kuramıyorlar.
- **Satır oyuncusu operayı**, **sütun oyuncusu futbolu** tercih ediyor; ama ikisi için de ayrı yerlere gitmek en kötüsü (0, 0).
- Standart savaşları (ör. iki firmanın farklı format desteklemesi) aynı yapıdadır.

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

Özellikler

- Asimetrik, sıfır toplamlı değil.
- Koordinasyon + çatışma: buluşmak ortak çıkar, *nerede* buluşulacağı çatışma.

KOORDİNASYON OYUNLARI

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Opera	Futbol
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

	sütun oyuncusu: Opera	sütun oyuncusu: Futbol
satır ↓ Opera	(3, 2)	(0, 0)
satır ↓ Futbol	(0, 0)	(2, 3)

Sütun oyuncusu **Opera** oynarsa satır oyuncusu: Opera → 3, Futbol → 0 ⇒ en iyi tepki **Opera**.

Adım 2 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Sütun oyuncusu **Futbol** oynarsa satır oyuncusu: Opera → 0, Futbol → 2 ⇒ en iyi tepki **Futbol**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

	sütun oyuncusu	
satır ↓	**Opera**	**Futbol**
Opera	(**3** , **2**)	(0 , 0)
Futbol	(0 , 0)	(**2** , 3)

Satır oyuncusu **Opera** oynarsa sütun oyuncusu: Opera → 2, Futbol → 0 ⇒ en iyi tepki **Opera**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(**3** , **2**)	(0 , 0)
Futbol	(0 , 0)	(**2** , **3**)

Satır oyuncusu **Futbol** oynarsa sütun oyuncusu: Opera → 0, Futbol → 3 ⇒ en iyi tepki **Futbol**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

İki işaretli hücre(ler): **(Opera, Opera), (Futbol, Futbol)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

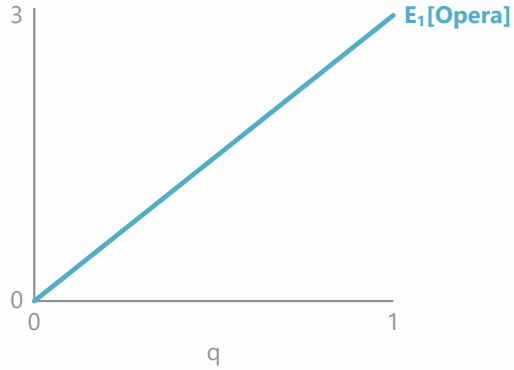
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3 \cdot q + 0 \cdot (1 - q) = 3q$



Sütun oyuncusu Opera'yı q olasılıkla oynasın. Satır oyuncusunun **Opera** ile beklenen kazancı q'ya göre bir doğrudur.

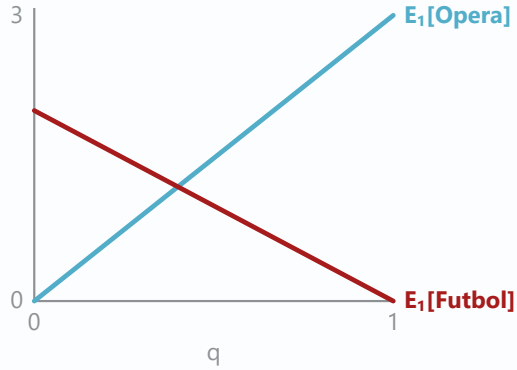
Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3 \cdot q + 0 \cdot (1 - q) = 3q$

$E_1[\text{Futbol}] = 0 \cdot q + 2 \cdot (1 - q) = -2q + 2$



Futbol için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

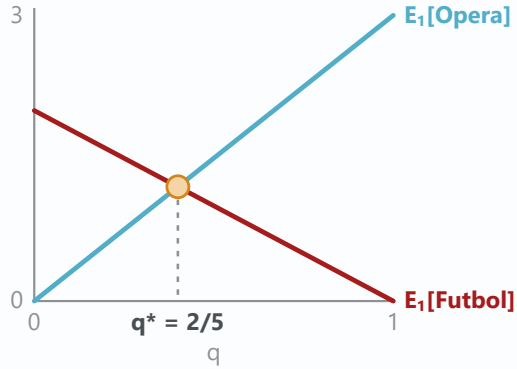
satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3\cdot q + 0\cdot(1 - q) = 3q$

$E_1[\text{Futbol}] = 0\cdot q + 2\cdot(1 - q) = -2q + 2$

Kayıtsızlık: $3q = -2q + 2 \Rightarrow 5q = 2 \Rightarrow \mathbf{q^* = 2/5}$



Karışık oynamak ancak iki saf strateji **eşit** kazandırıyorsa mantıklıdır: kesişim q* = 2/5.

Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3\cdot q + 0\cdot(1-q) = 3q$

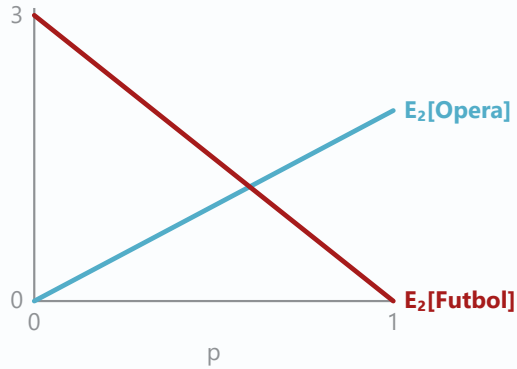
$E_1[\text{Futbol}] = 0\cdot q + 2\cdot(1-q) = -2q + 2$

Kayıtsızlık: $3q = -2q + 2 \Rightarrow 5q = 2 \Rightarrow \mathbf{q^* = 2/5}$

p = P(satır oyuncusu **Opera** oynar)

$E_2[\text{Opera}] = 2\cdot p + 0\cdot(1-p) = 2p$

$E_2[\text{Futbol}] = 0\cdot p + 3\cdot(1-p) = -3p + 3$



Aynı mantık sütun oyuncusu için: satır oyuncusu Opera'yı p olasılıkla oynasın; sütunun iki doğrusu.

Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3\cdot q + 0\cdot(1-q) = 3q$

$E_1[\text{Futbol}] = 0\cdot q + 2\cdot(1-q) = -2q + 2$

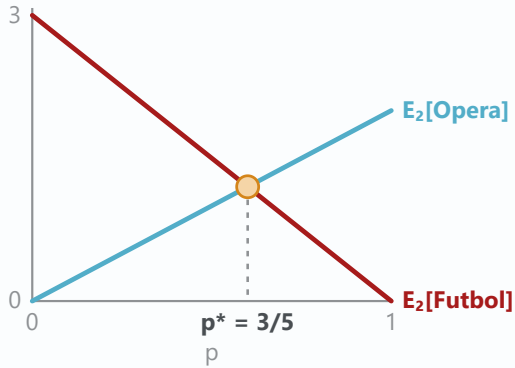
Kayıtsızlık: $3q = -2q + 2 \Rightarrow 5q = 2 \Rightarrow \mathbf{q^* = 2/5}$

p = P(satır oyuncusu **Opera** oynar)

$E_2[\text{Opera}] = 2\cdot p + 0\cdot(1-p) = 2p$

$E_2[\text{Futbol}] = 0\cdot p + 3\cdot(1-p) = -3p + 3$

Kayıtsızlık: $2p = -3p + 3 \Rightarrow 5p = 3 \Rightarrow \mathbf{p^* = 3/5}$



Satır oyuncusu p* = 3/5 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Cinsiyetler Savaşı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Opera	Futbol
Opera	(3, 2)	(0, 0)
Futbol	(0, 0)	(2, 3)

q = P(sütun oyuncusu **Opera** oynar)

$E_1[\text{Opera}] = 3\cdot q + 0\cdot(1-q) = 3q$

$E_1[\text{Futbol}] = 0\cdot q + 2\cdot(1-q) = -2q + 2$

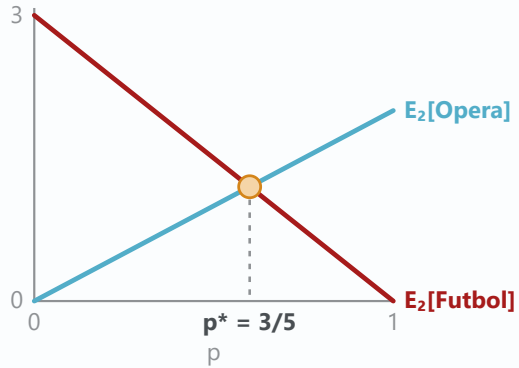
Kayıtsızlık: $3q = -2q + 2 \Rightarrow 5q = 2 \Rightarrow \mathbf{q^* = 2/5}$

p = P(satır oyuncusu **Opera** oynar)

$E_2[\text{Opera}] = 2\cdot p + 0\cdot(1-p) = 2p$
$E_2[\text{Futbol}] = 0\cdot p + 3\cdot(1-p) = -3p + 3$

Kayıtsızlık: $2p = -3p + 3 \Rightarrow 5p = 3 \Rightarrow \mathbf{p^* = 3/5}$

Denge: satır (Opera 3/5, Futbol 2/5), sütun (Opera 2/5, Futbol 3/5) · beklenen kazanç $u_1 = 6/5$, $u_2 = 6/5$



Sonuç: her oyuncu rakibini kayıtsız bırakan karışımı oynar; kimse tek başına sapmakla kazanamaz.

Kod İzleme: `karisik_2x2(A, B)`

satır \ sütun oyuncusu	Opera	Futbol
Opera	(3 , 2)	(0 , 0)
Futbol	(0 , 0)	(2 , 3)

Cinsiyetler Savaşı: A satır, B sütun oyuncusunun kazançları (0 = Opera, 1 = Futbol).

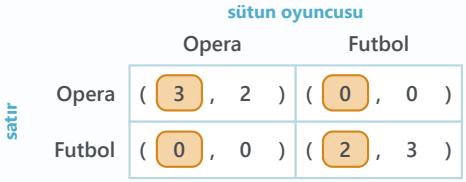
Adım 1 / 8

```
1  from fractions import Fraction as F
▶  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

—

Kod İzleme: `karisik_2x2(A, B)`



Satır oyuncusunun kazanç farklarından payda: da = 3 − 0 − 0 + 2 = **5**.

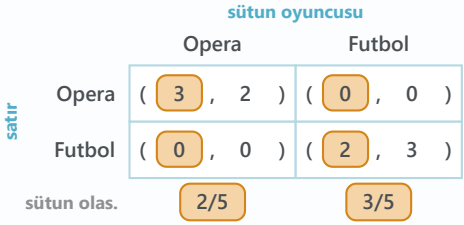
Adım 2 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
▶      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5`

Kod İzleme: `karisik_2x2(A, B)`



E_1[Opera] = 3q, E_1[Futbol] = 2(1 − q) eşitlenir ⇒ q = (2 − 0)/5 = **2/5**: sütun oyuncusu Opera'yı 2/5 olasılıkla oynar.

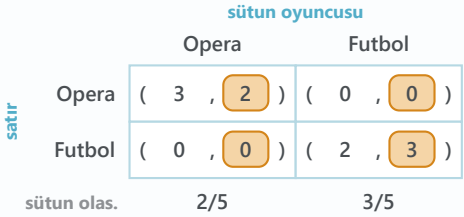
Adım 3 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
▶      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5` `q = 2/5`

Kod İzleme: `karisik_2x2(A, B)`



Şimdi sütun oyuncusunun kazançları: db = 2 − 0 − 0 + 3 = **5**.

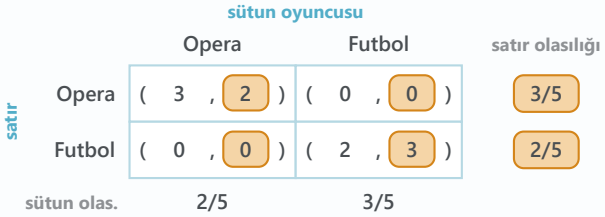
Adım 4 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
▶      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5` `q = 2/5` `db = 5`

Kod İzleme: `karisik_2x2(A, B)`



E_2[Opera] = 2p, E_2[Futbol] = 3(1 − p) eşitlenir ⇒ p = **3/5**: satır oyuncusu Opera'yı 3/5 olasılıkla oynar.

Adım 5 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
▶      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5` `q = 2/5` `db = 5` `p = 3/5`

Kod İzleme: `karisik_2x2(A, B)`

satır \ sütun oyuncusu	Opera	Futbol	satır olasılığı
Opera	(3 , 2)	(0 , 0)	3/5
Futbol	(0 , 0)	(2 , 3)	2/5
sütun olas.	2/5	3/5	

0 < p, q < 1 doğru: iki olasılık da gerçek bir karışım ⇒ iç karışık denge var, return None atlanır.

Adım 6 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
▶      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
9      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5` `q = 2/5` `db = 5` `p = 3/5`

Kod İzleme: `karisik_2x2(A, B)`

		sütun oyuncusu: Opera	sütun oyuncusu: Futbol	satır olasılığı
satır	Opera	(**3** , 2)	(**0** , 0)	3/5
	Futbol	(0 , 0)	(2 , 3)	2/5
	sütun olas.	2/5	3/5	

Beklenen kazanç $u_1 = 3\cdot(2/5) + 0\cdot(3/5) = \mathbf{6/5}$; $u_2 = 2\cdot(3/5) = 6/5$. İki saf dengenin (2 ya da 3) ikisinden de kötü.

Adım 7 / 8

```
1  from fractions import Fraction as F
2  def karisik_2x2(A, B):
3      da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4      q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5      db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6      p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7      if not (0 < p < 1 and 0 < q < 1):
8          return None               # iç denge yok
▶      u1 = A[0][0] * q + A[0][1] * (1 - q)
10     return p, q, u1
```

DEĞİŞKENLER

`da = 5` `q = 2/5` `db = 5` `p = 3/5` `u1 = 6/5`

Kod İzleme: `karisik_2x2(A, B)`

satır \ sütun oyuncusu	Opera	Futbol	satır olasılığı
Opera	(3 , 2)	(0 , 0)	3/5
Futbol	(0 , 0)	(2 , 3)	2/5
sütun olas.	2/5	3/5	

Sonuç $(p, q, u_1) = (3/5, 2/5, 6/5)$. Aynı fonksiyon Bedava Para'da $q = 0$ bulur ve None döndürür: iç karışık denge yok.

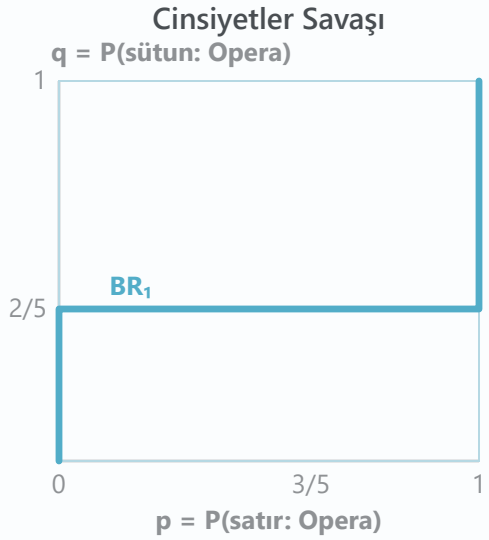
Adım 8 / 8

```
1 from fractions import Fraction as F
2 def karisik_2x2(A, B):
3     da = A[0][0] - A[0][1] - A[1][0] + A[1][1]
4     q = F(A[1][1] - A[0][1], da)  # satır kayıtsız
5     db = B[0][0] - B[0][1] - B[1][0] + B[1][1]
6     p = F(B[1][1] - B[1][0], db)  # sütun kayıtsız
7     if not (0 < p < 1 and 0 < q < 1):
8         return None               # iç denge yok
9     u1 = A[0][0] * q + A[0][1] * (1 - q)
▶     return p, q, u1
```

DEĞİŞKENLER

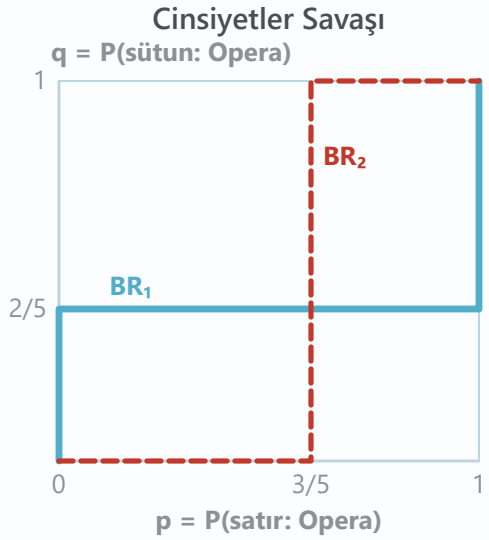
`da = 5` `q = 2/5` `db = 5` `p = 3/5` `u1 = 6/5`

En İyi Tepki Eğrileri: Dengeler Kesişimdedir



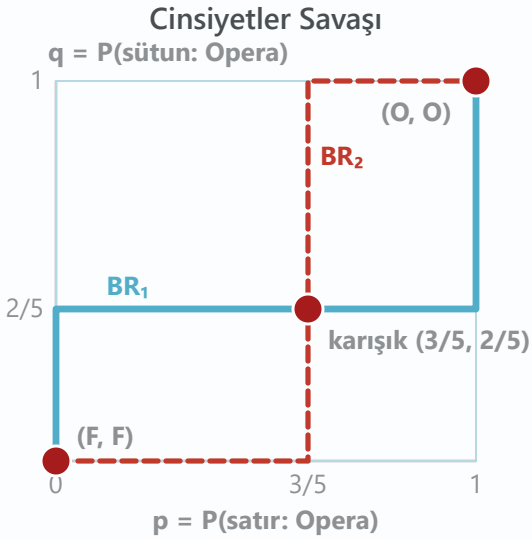
$BR_1(q)$: $q < 2/5$ iken satır Futbol'u ($p = 0$), $q > 2/5$ iken Opera'yı ($p = 1$) seçer; tam $q = 2/5$'te her p en iyi (dikey değil, **yatay** parça).

En İyi Tepki Eğrileri: Dengeler Kesişimdedir



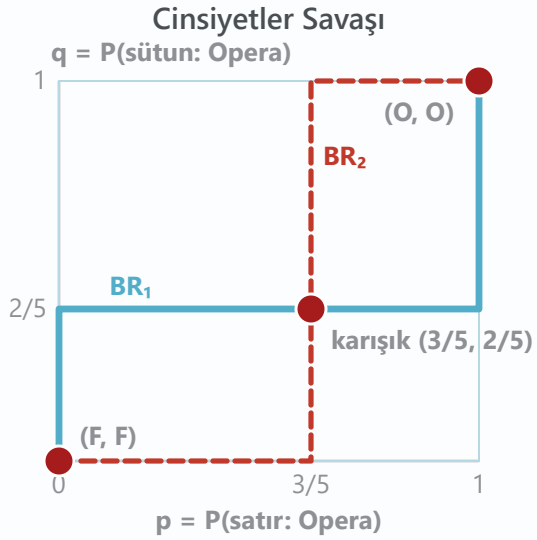
$BR_2(p)$: $p < 3/5$ iken sütun Futbol'u ($q = 0$), $p > 3/5$ iken Opera'yı ($q = 1$) seçer; $p = 3/5$'te kayıtsız.

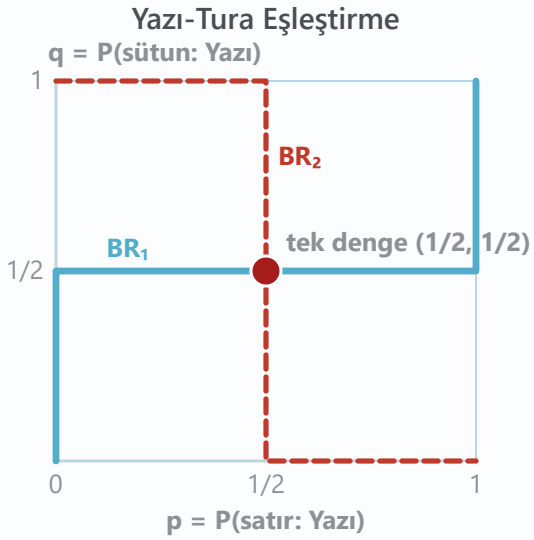
En İyi Tepki Eğrileri: Dengeler Kesişimdedir



Nash dengesi = iki eğrinin **kesişimi**: (0, 0) = (Futbol, Futbol), (3/5, 2/5) karışık, (1, 1) = (Opera, Opera). Üç denge.

En İyi Tepki Eğrileri: Dengeler Kesişimdedir





Yazı-Tura'da Ayrıştıranın eğrisi ters yöndedir; eğriler yalnızca (1/2, 1/2)'de kesişir: saf denge yok, tek karışık denge.

Cinsiyetler Savaşı: Sonuçların Yorumu

P(sonuç)	Opera (2/5)	Futbol (3/5)
Opera (3/5)	6/25 = %24 (3, 2)	9/25 = %36 (0, 0)
Futbol (2/5)	4/25 = %16 (0, 0)	6/25 = %24 (2, 3)

Turuncu: buluşamama, toplam 13/25 = %52

Kim ne oynar?

Satır (operasever) Opera'yı %60, sütun (futbolsever) Futbol'u %60 oynar.

Buluşamama

P = 0,36 + 0,16 = **%52**.

Beklenen kazanç

$u_1 = u_2 = 6/5 = 1{,}2$; saf dengelerde herkes en az 2 alır.

Karışık denge çatışmayı **çözmez**: %52 ihtimalle buluşamazlar ve 6/5 ile ikisi de her iki saf dengeden kötüdür. Çözüm oyunun dışından gelir: iletişim, gelenek, sırayla seçmek.

Geyik Avı (Stag Hunt)

- İki avcı: **geyik** ancak ikisi birlikte avlanırsa yakalanır (2, 2); **tavşanı** tek başına kesin yakalarsın (1).
- Tek başına geyik peşine düşen eli boş kalır (0).
- Rousseau'nun örneği; ortak projeler, iklim anlaşmaları, yeni teknolojiye birlikte geçiş.

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

Özellikler

- Simetrik, sıfır toplamlı değil.
- (Geyik, Geyik) Pareto optimal; (Tavşan, Tavşan) güvenli.

KOORDİNASYON OYUNLARI

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(2 , 2)	(0 , 1)
Tavşan	(1 , 0)	(1 , 1)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(2 , 2)	(0 , 1)
Tavşan	(1 , 0)	(1 , 1)

Sütun oyuncusu **Geyik** oynarsa satır oyuncusu: Geyik → 2, Tavşan → 1 ⇒ en iyi tepki **Geyik**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(**2** , 2)	(0 , 1)
Tavşan	(1 , 0)	(**1** , 1)

Sütun oyuncusu **Tavşan** oynarsa satır oyuncusu: Geyik → 0, Tavşan → 1 ⇒ en iyi tepki **Tavşan**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Geyik	sütun oyuncusu: Tavşan
Geyik	(**2** , **2**)	(0 , 1)
Tavşan	(1 , 0)	(**1** , 1)

Satır oyuncusu **Geyik** oynarsa sütun oyuncusu: Geyik → 2, Tavşan → 1 ⇒ en iyi tepki **Geyik**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Geyik	Tavşan
Geyik	(**2** , **2**)	(0 , 1)
Tavşan	(1 , 0)	(**1** , **1**)

Satır oyuncusu **Tavşan** oynarsa sütun oyuncusu: Geyik → 0, Tavşan → 1 ⇒ en iyi tepki **Tavşan**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Geyik	sütun oyuncusu: Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

İki işaretli hücre(ler): **(Geyik, Geyik), (Tavşan, Tavşan)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

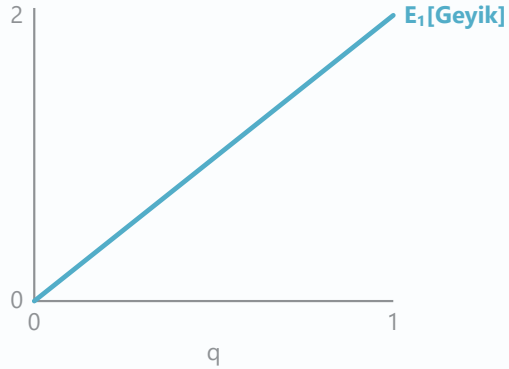
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Geyik Avı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

$E_1[\text{Geyik}] = 2 \cdot q + 0 \cdot (1 - q) = 2q$



Sütun oyuncusu Geyik'i q olasılıkla oynasın. Satır oyuncusunun **Geyik** ile beklenen kazancı q'ya göre bir doğrudur.

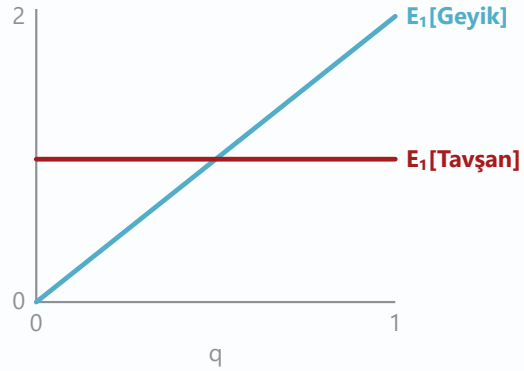
Geyik Avı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

$E_1[\text{Geyik}] = 2 \cdot q + 0 \cdot (1 - q) = 2q$

$E_1[\text{Tavşan}] = 1 \cdot q + 1 \cdot (1 - q) = 1$



Tavşan için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Geyik Avı: Karışık Denge (Kayıtsızlık)

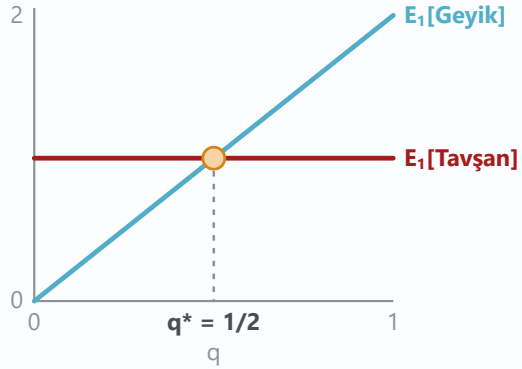
satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

$E_1[\text{Geyik}] = 2 \cdot q + 0 \cdot (1 - q) = 2q$

$E_1[\text{Tavşan}] = 1 \cdot q + 1 \cdot (1 - q) = 1$

Kayıtsızlık: $2q = 1 \Rightarrow \mathbf{q^* = 1/2}$



Karışık oynamak ancak iki saf strateji **eşit** kazandırıyorsa mantıklıdır: kesişim q* = 1/2.

Geyik Avı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

E_1[Geyik] = 2·q + 0·(1 − q) = 2q

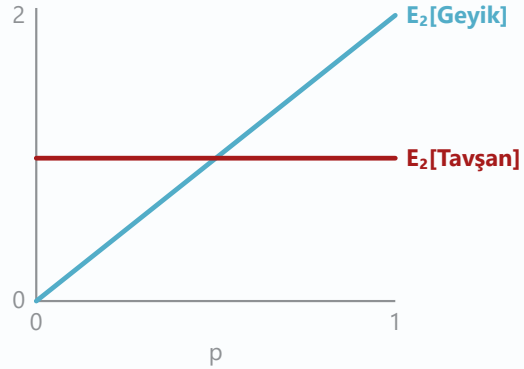
E_1[Tavşan] = 1·q + 1·(1 − q) = 1

Kayıtsızlık: 2q = 1 ⇒ **q* = 1/2**

p = P(satır oyuncusu **Geyik** oynar)

E_2[Geyik] = 2·p + 0·(1 − p) = 2p

E_2[Tavşan] = 1·p + 1·(1 − p) = 1



Aynı mantık sütun oyuncusu için: satır oyuncusu Geyik'i p olasılıkla oynasın; sütunun iki doğrusu.

Geyik Avı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

E_1[Geyik] = 2·q + 0·(1 − q) = 2q

E_1[Tavşan] = 1·q + 1·(1 − q) = 1

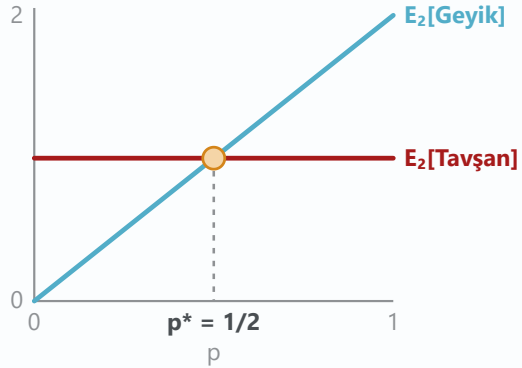
Kayıtsızlık: 2q = 1 ⇒ **q* = 1/2**

p = P(satır oyuncusu **Geyik** oynar)

E_2[Geyik] = 2·p + 0·(1 − p) = 2p

E_2[Tavşan] = 1·p + 1·(1 − p) = 1

Kayıtsızlık: 2p = 1 ⇒ **p* = 1/2**



Satır oyuncusu p* = 1/2 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Geyik Avı: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Geyik	Tavşan
Geyik	(2, 2)	(0, 1)
Tavşan	(1, 0)	(1, 1)

q = P(sütun oyuncusu **Geyik** oynar)

E_1[Geyik] = 2·q + 0·(1 − q) = 2q

E_1[Tavşan] = 1·q + 1·(1 − q) = 1

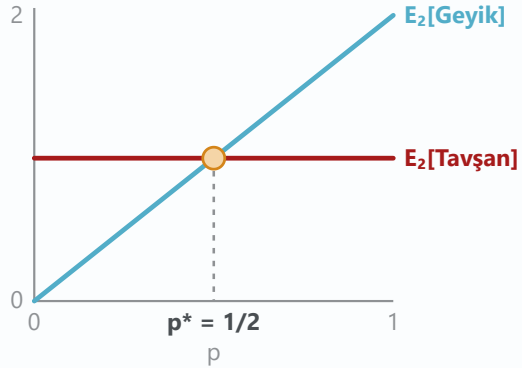
Kayıtsızlık: 2q = 1 ⇒ **q* = 1/2**

p = P(satır oyuncusu **Geyik** oynar)

E_2[Geyik] = 2·p + 0·(1 − p) = 2p
E_2[Tavşan] = 1·p + 1·(1 − p) = 1

Kayıtsızlık: 2p = 1 ⇒ **p* = 1/2**

Denge: satır (Geyik 1/2, Tavşan 1/2), sütun (Geyik 1/2, Tavşan 1/2) · beklenen kazanç u_1 = 1, u_2 = 1



Sonuç: her oyuncu rakibini kayıtsız bırakan karışımı oynar; kimse tek başına sapmakla kazanamaz.

Geyik Avı: Sonuçların Yorumu

P(sonuç)	Geyik (1/2)	Tavşan (1/2)
Geyik (1/2)	1/4 = %25 (2, 2)	1/4 = %25 (0, 1)
Tavşan (1/2)	1/4 = %25 (1, 0)	1/4 = %25 (1, 1)

Turuncu: avcılar ayrı düşer, toplam %50

Beklenen kazanç

$u_1 = u_2 = 1$: tam olarak tavşanın garantili kazancı kadar.

Ayrı düşme

P = 1/4 + 1/4 = %50; ikisi birlikte geyik: yalnızca %25.

Eşik inanç

Karşı tarafın geyik avlama olasılığı ≥ 1/2 ise geyik avlamak en iyi tepkidir.

Bu bir **güven ikilemi**dir: (Geyik, Geyik) herkes için en iyisi ve bir denge, ama karşı tarafa güvenmeyen için Tavşan risksizdir. Mahkûmun İkilemi'nden farkı: burada işbirliği **dengedir**; sorun çıkar çatışması değil, güven eksikliğidir.

Saf Koordinasyon (Pure Coordination)

- İki arkadaş akşam **partiye** mi gidecek, **evde** mi kalacak? İletişim yok.
- İkisi de aynı şeyi yapmak ister; birlikte parti (10, 10) birlikte evden (5, 5) daha iyi.
- Farklı karar verirlerse ikisi de 0 alır: evde kalan da parti kaçıran da yalnız kalır.

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

Özellikler

- Simetrik; **ortak çıkar**: her hücrede $u_1 = u_2$.
- İki saf denge Pareto sıralı: (Parti, Parti) daha iyi.

KOORDİNASYON OYUNLARI

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	sütun oyuncusu: Parti	sütun oyuncusu: Ev
Parti	(10 , 10)	(0 , 0)
Ev	(0 , 0)	(5 , 5)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , 10)	(0 , 0)
Ev	(0 , 0)	(5 , 5)

Sütun oyuncusu **Parti** oynarsa satır oyuncusu: Parti → 10, Ev → 0 ⇒ en iyi tepki **Parti**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , 10)	(0 , 0)
Ev	(0 , 0)	(**5** , 5)

Sütun oyuncusu **Ev** oynarsa satır oyuncusu: Parti → 0, Ev → 5 ⇒ en iyi tepki **Ev**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , **10**)	(0 , 0)
Ev	(0 , 0)	(**5** , 5)

Satır oyuncusu **Parti** oynarsa sütun oyuncusu: Parti → 10, Ev → 0 ⇒ en iyi tepki **Parti**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle

satır ↓	Parti (sütun oyuncusu)	Ev (sütun oyuncusu)
Parti	(**10** , **10**)	(0 , 0)
Ev	(0 , 0)	(**5** , **5**)

Satır oyuncusu **Ev** oynarsa sütun oyuncusu: Parti → 0, Ev → 5 ⇒ en iyi tepki **Ev**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Saf Dengeleri İşaretle



İki işaretli hücre(ler): **(Parti, Parti), (Ev, Ev)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

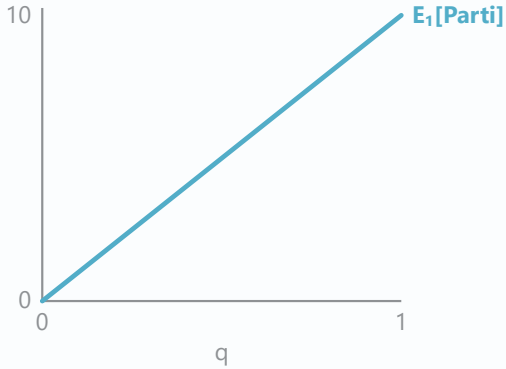
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$



Sütun oyuncusu Parti'yi q olasılıkla oynasın. Satır oyuncusunun **Parti** ile beklenen kazancı q'ya göre bir doğrudur.

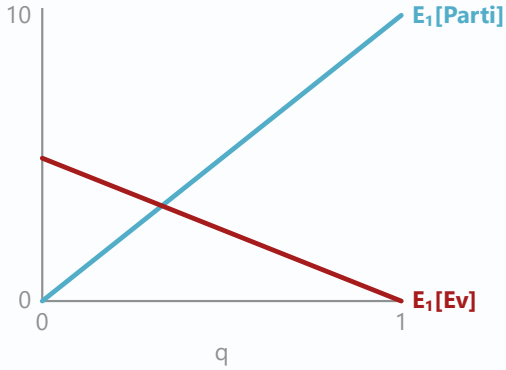
Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$

$E_1[\text{Ev}] = 0 \cdot q + 5 \cdot (1 - q) = -5q + 5$



Ev için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

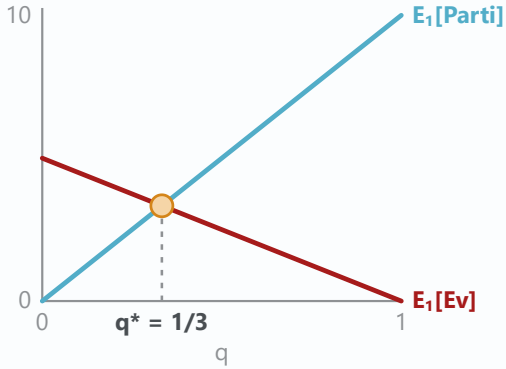
satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$

$E_1[\text{Ev}] = 0 \cdot q + 5 \cdot (1 - q) = -5q + 5$

Kayıtsızlık: $10q = -5q + 5 \Rightarrow 15q = 5 \Rightarrow \mathbf{q^* = 1/3}$



Karışık oynamak ancak iki saf strateji **eşit** kazandırıyorsa mantıklıdır: kesişim q* = 1/3.

Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$

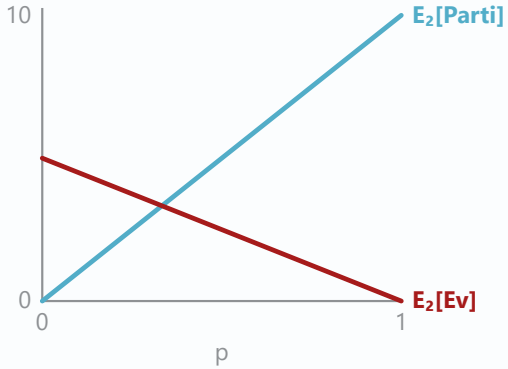
$E_1[\text{Ev}] = 0 \cdot q + 5 \cdot (1 - q) = -5q + 5$

Kayıtsızlık: $10q = -5q + 5 \Rightarrow 15q = 5 \Rightarrow \mathbf{q^* = 1/3}$

p = P(satır oyuncusu **Parti** oynar)

$E_2[\text{Parti}] = 10 \cdot p + 0 \cdot (1 - p) = 10p$

$E_2[\text{Ev}] = 0 \cdot p + 5 \cdot (1 - p) = -5p + 5$



Aynı mantık sütun oyuncusu için: satır oyuncusu Parti'yi p olasılıkla oynasın; sütunun iki doğrusu.

Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$

$E_1[\text{Ev}] = 0 \cdot q + 5 \cdot (1 - q) = -5q + 5$

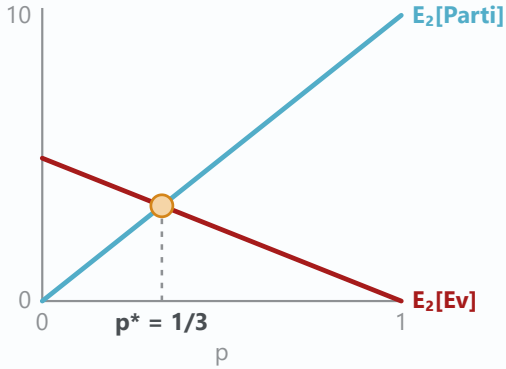
Kayıtsızlık: $10q = -5q + 5 \Rightarrow 15q = 5 \Rightarrow \mathbf{q^* = 1/3}$

p = P(satır oyuncusu **Parti** oynar)

$E_2[\text{Parti}] = 10 \cdot p + 0 \cdot (1 - p) = 10p$

$E_2[\text{Ev}] = 0 \cdot p + 5 \cdot (1 - p) = -5p + 5$

Kayıtsızlık: $10p = -5p + 5 \Rightarrow 15p = 5 \Rightarrow \mathbf{p^* = 1/3}$



Satır oyuncusu p* = 1/3 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Saf Koordinasyon: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Parti	Ev
Parti	(10, 10)	(0, 0)
Ev	(0, 0)	(5, 5)

q = P(sütun oyuncusu **Parti** oynar)

$E_1[\text{Parti}] = 10 \cdot q + 0 \cdot (1 - q) = 10q$

$E_1[\text{Ev}] = 0 \cdot q + 5 \cdot (1 - q) = -5q + 5$

Kayıtsızlık: $10q = -5q + 5 \Rightarrow 15q = 5 \Rightarrow \mathbf{q^* = 1/3}$

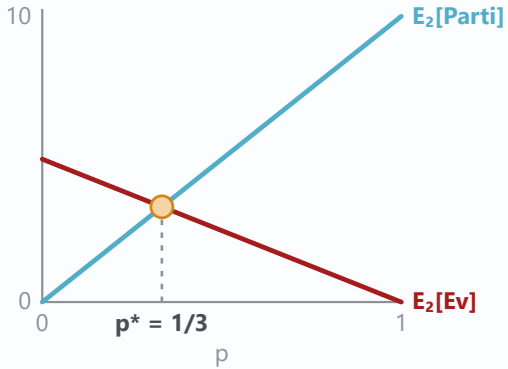
p = P(satır oyuncusu **Parti** oynar)

$E_2[\text{Parti}] = 10 \cdot p + 0 \cdot (1 - p) = 10p$

$E_2[\text{Ev}] = 0 \cdot p + 5 \cdot (1 - p) = -5p + 5$

Kayıtsızlık: $10p = -5p + 5 \Rightarrow 15p = 5 \Rightarrow \mathbf{p^* = 1/3}$

Denge: satır (Parti 1/3, Ev 2/3), sütun (Parti 1/3, Ev 2/3) · beklenen kazanç $u_1 = 10/3$, $u_2 = 10/3$



Sonuç: her oyuncu rakibini kayıtsız bırakan karışımı oynar; kimse tek başına sapmakla kazanamaz.

Saf Koordinasyon: Sonuçların Yorumu

P(sonuç)	Parti (1/3)	Ev (2/3)
Parti (1/3)	1/9 = %11,1 (10, 10)	2/9 = %22,2 (0, 0)
Ev (2/3)	2/9 = %22,2 (0, 0)	4/9 = %44,4 (5, 5)

Turuncu: farklı karar, toplam 4/9 ≈ %44,4

Beklenen kazanç

$u_1 = u_2 = 10/3 \approx 3{,}33$: **iki saf dengeden de düşük** (10 ve 5).

Buluşamama

$P = 2 \cdot 1/3 \cdot 2/3 = 4/9 \approx \%44{,}4$.

Evde kalmak "garantili" değil

Karşı taraf partiye giderse evde kalan 0 alır. Güvenli strateji yoktur.

Karışık dengede parti olasılığı yalnızca 1/3: parti ne kadar cazipse rakibi kayıtsız bırakmak için o kadar *az* seçilir. Gerçekte oyuncular bir **odak noktasına** (Schelling) yönelir: (Parti, Parti) hem Pareto hem de belirgin olduğu için seçilir.

Bedava Para (Free Money)

- Her iki oyuncu **Yazı** derse ikisine de 1 TL verilir; diğer üç sonuçta kimse bir şey kazanmaz.
- Masada "bedava para" var; tek gereken ikisinin de Yazı demesi.
- Basit görünür ama iki farklı denge türünü ayırt etmeyi öğretir.

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Özellikler

- Simetrik, ortak çıkar.
- Yazı **zayıf baskın** strateji: hiç kötü değil, bazen daha iyi.

KOORDİNASYON OYUNLARI

Bedava Para: Saf Dengeleri İşaretle

sütun oyuncusu

satır ↓	Yazı	Tura
Yazı	(1 , 1)	(0 , 0)
Tura	(0 , 0)	(0 , 0)

Satır oyuncusunun her sütuna, sütun oyuncusunun her satıra **en iyi tepkisini** (best response) işaretleyeceğiz. İki işaretli hücre = saf Nash dengesi.

Adım 1 / 6

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(1 , 1)	(0 , 0)
Tura	(0 , 0)	(0 , 0)

Sütun oyuncusu **Yazı** oynarsa satır oyuncusu: Yazı → 1, Tura → 0 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

	sütun oyuncusu: Yazı	sütun oyuncusu: Tura
satır ↓ Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Sütun oyuncusu **Tura** oynarsa satır oyuncusu: Yazı → 0, Tura → 0 ⇒ eşitlik: **ikisi de** en iyi tepki, ikisini de işaretle.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓	Yazı	Tura
	sütun oyuncusu	
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Satır oyuncusu **Yazı** oynarsa sütun oyuncusu: Yazı → 1, Tura → 0 ⇒ en iyi tepki **Yazı**.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle

satır ↓ \ sütun oyuncusu	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

Satır oyuncusu **Tura** oynarsa sütun oyuncusu: Yazı → 0, Tura → 0 ⇒ eşitlik: **ikisi de** en iyi tepki.

Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Saf Dengeleri İşaretle



İki işaretli hücre(ler): **(Yazı, Yazı), (Tura, Tura)** ⇒ 2 saf Nash dengesi. Hiçbir oyuncu tek başına sapmakla kazanamaz.

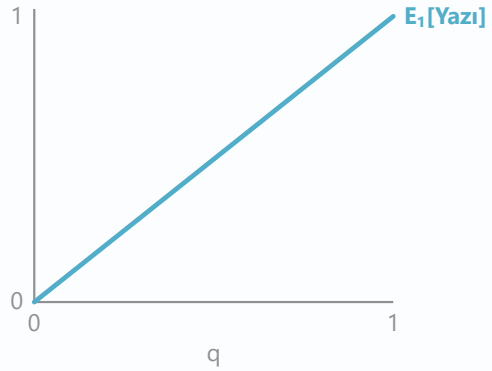
Gösterim: turuncu = en iyi tepki · mavi zemin = incelenen sütun/satır · kırmızı çerçeve = saf Nash dengesi. Hücrede (satır oyuncusunun, sütun oyuncusunun) kazancı.

Bedava Para: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + 0 \cdot (1 - q) = q$



Sütun oyuncusu Yazı'yı q olasılıkla oynasın. Satır oyuncusunun **Yazı** ile beklenen kazancı q'ya göre bir doğrudur.

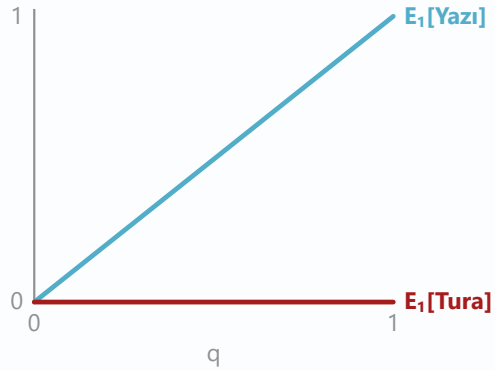
Bedava Para: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + 0 \cdot (1 - q) = q$

$E_1[\text{Tura}] = 0 \cdot q + 0 \cdot (1 - q) = 0$



Tura için de doğruyu çiz. Hangi doğru üstteyse satır oyuncusunun en iyi tepkisi odur.

Bedava Para: Karışık Denge (Kayıtsızlık)

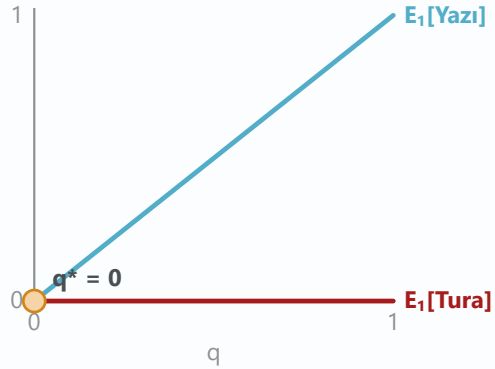
satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + 0 \cdot (1 - q) = q$

$E_1[\text{Tura}] = 0 \cdot q + 0 \cdot (1 - q) = 0$

Kayıtsızlık: $q = 0 \Rightarrow \mathbf{q^* = 0}$



Doğrular yalnızca q = 0'da kesişir: q > 0 ise Yazı kesinlikle daha iyi.

Bedava Para: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + 0 \cdot (1 - q) = q$

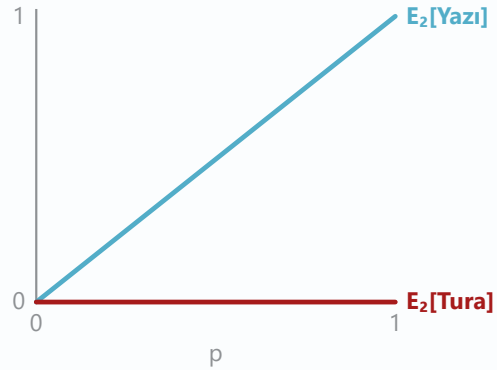
$E_1[\text{Tura}] = 0 \cdot q + 0 \cdot (1 - q) = 0$

Kayıtsızlık: $q = 0 \Rightarrow \mathbf{q^* = 0}$

p = P(satır oyuncusu **Yazı** oynar)

$E_2[\text{Yazı}] = 1 \cdot p + 0 \cdot (1 - p) = p$

$E_2[\text{Tura}] = 0 \cdot p + 0 \cdot (1 - p) = 0$



Aynı mantık sütun oyuncusu için: satır oyuncusu Yazı'yı p olasılıkla oynasın; sütunun iki doğrusu.

Bedava Para: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

$E_1[\text{Yazı}] = 1 \cdot q + 0 \cdot (1 - q) = q$

$E_1[\text{Tura}] = 0 \cdot q + 0 \cdot (1 - q) = 0$

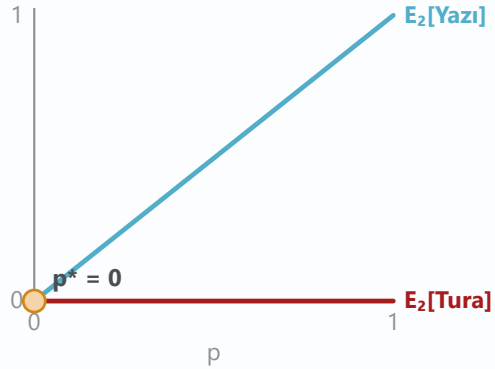
Kayıtsızlık: $q = 0 \Rightarrow \mathbf{q^* = 0}$

p = P(satır oyuncusu **Yazı** oynar)

$E_2[\text{Yazı}] = 1 \cdot p + 0 \cdot (1 - p) = p$

$E_2[\text{Tura}] = 0 \cdot p + 0 \cdot (1 - p) = 0$

Kayıtsızlık: $p = 0 \Rightarrow \mathbf{p^* = 0}$



Satır oyuncusu p* = 0 seçerek sütun oyuncusunu kayıtsız bırakır. Dikkat: p*, satırın kendi kazancından değil **rakibin** kazancından çıkar.

Bedava Para: Karışık Denge (Kayıtsızlık)

satır ╲ sütun	Yazı	Tura
Yazı	(1, 1)	(0, 0)
Tura	(0, 0)	(0, 0)

q = P(sütun oyuncusu **Yazı** oynar)

E_1[Yazı] = 1·q + 0·(1 − q) = q

E_1[Tura] = 0·q + 0·(1 − q) = 0

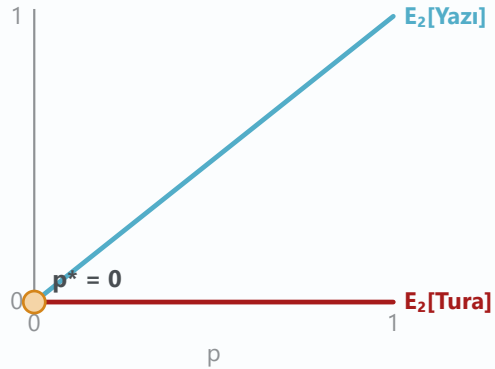
Kayıtsızlık: q = 0 ⇒ **q* = 0**

p = P(satır oyuncusu **Yazı** oynar)

E_2[Yazı] = 1·p + 0·(1 − p) = p
E_2[Tura] = 0·p + 0·(1 − p) = 0

Kayıtsızlık: p = 0 ⇒ **p* = 0**

p* = q* = 0: ikisi de saf **Tura** ⇒ bu, (Tura, Tura) saf dengesidir; tam karışık denge yok.



Kayıtsızlık yalnızca uç noktada sağlanıyor ⇒ tam karışık (iki stratejiye de pozitif olasılık veren) denge yoktur.

Bedava Para: Sonuçların Yorumu

P(sonuç)	Yazı (0)	Tura (1)
Yazı (0)	0 = %0 (1, 1)	0 = %0 (0, 0)
Tura (1)	0 = %0 (0, 0)	1 = %100 (0, 0)

p* = q* = 0: tüm olasılık (Tura, Tura) hücresinde

Kesin denge (strict): (Yazı, Yazı)

Sapan oyuncu kesin kaybeder (1 → 0).

Zayıf denge: (Tura, Tura)

Sapmak kazandırmaz ama kaybettirmez de (0 → 0); karşı taraf Tura derken Yazı demek boşunadır.

Karışık denge yok

q > 0 ise Yazı kesin daha iyidir; kayıtsızlık yalnızca q = 0'da.

Nash dengeleri çoğu oyunda **tek sayıdadır** (Wilson, 1971: "tek sayılılık teoremi"). Bedava Para'da 2 denge vardır, çünkü oyun **dejeneredir** (eşit kazançlar içerir) ve teoremin varsayımları sağlanmaz. Zayıf baskın stratejileri elemek (Tura'yı atmak) mantıklı dengeyi, (Yazı, Yazı)'yı seçer.

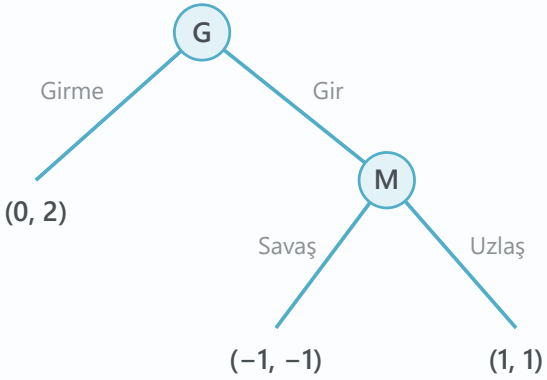
Alt Oyun ve Alt Oyun Mükemmel Denge

- **Alt oyun** (subgame): tek bir karar düğümünden başlayıp altındaki her şeyi içeren (bilgi kümelerini bölmeyen) oyun parçası.
- **Alt oyun mükemmel denge** (subgame perfect equilibrium, SPE): **her alt oyunda** Nash dengesi oluşturan strateji profili (Selten, 1965).
- Sıradan Nash dengesi, hiç ulaşılmayan düğümlerde **inandırıcı olmayan tehditlere** izin verebilir; SPE bunları eler.
- Sonlu ve mükemmel bilgili oyunlarda SPE **geriye doğru çıkarımla** (backward induction) bulunur (Kuhn/Zermelo).

Geriye doğru çıkarım

1. Yalnızca yapraklara bağlı son karar düğümlerini bul.
2. Orada karar veren oyuncunun en iyi dalını seç.
3. Düğümü, seçilen yaprağın kazancıyla değiştir.
4. Köke ulaşana kadar tekrarla.

Örnek: İnandırıcı Olmayan Tehdit



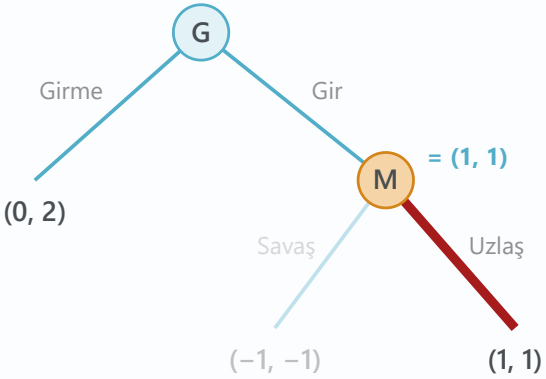
G önce seçer; M, yalnızca G girerse karar verir. Normal formda iki Nash dengesi vardır (sağdaki tablo).

Adım 1 / 3

	sütun oyuncusu: Savaş	sütun oyuncusu: Uzlaş
satır ↓ Gir	(−1 , −1)	(1 , 1)
satır ↓ Girme	(0 , 2)	(0 , 2)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

Örnek: İnandırıcı Olmayan Tehdit

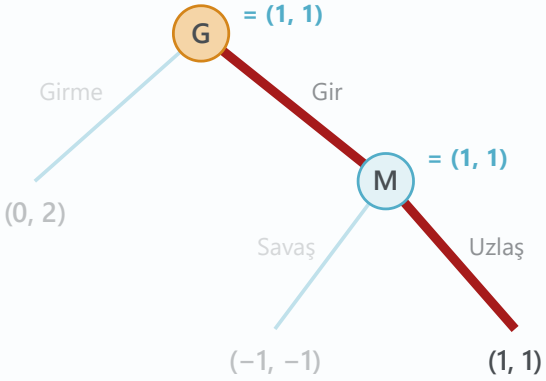


Son düğüm M: Savaş → −1, Uzlaş → 1 ⇒ **Uzlaş**. M'nin düğümünün değeri (1, 1).

satır ↓ \ sütun oyuncusu	Savaş	Uzlaş
Gir	(−1 , −1)	(1 , 1)
Girme	(0 , 2)	(0 , 2)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

Örnek: İnandırıcı Olmayan Tehdit



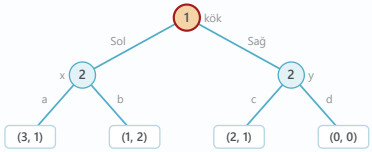
Kök G: Girme → 0, Gir → 1 (M'nin uzlaşacağını bilerek) ⇒ **Gir**. SPE: (Gir, Uzlaş), sonuç (1, 1).

sütun oyuncusu

satır ↓	Savaş	Uzlaş
Gir	(−1 , −1)	(1 , 1)
Girme	(0 , 2)	(0 , 2)

- (Gir, Uzlaş): SPE, geriye doğru çıkarımın sonucu.
- (Girme, Savaş): Nash dengesi, ama M'nin "girersen savaşırım" tehdidi **inandırıcı değil**: G girdiğinde savaşmak M'ye −1, uzlaşmak 1 verir.

Kod İzleme: `cozum(d)`



Kökte oyuncu 1 (indis 0), x ve y'de oyuncu 2 (indis 1) seçer. cozum(kök) çağrılır.

Adım 1 / 10

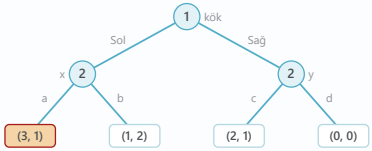
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
▶      ad, oyuncu, dallar = d
▶      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
▶  print(cozum(kok))
```

DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `en_iyi = None`

ÇIKTI

Kod İzleme: `cozum(d)`



Sol dalı: cozum(x) → ilk dal a, yaprak (3, 1) olduğu gibi döner.

Adım 2 / 10

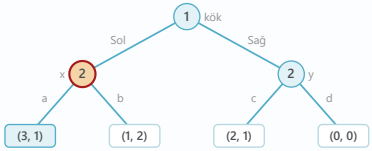
```
1  def cozum(d):
▶      if isinstance(d, tuple):          # yaprak: kazanç
▶          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `hamle = a` `v = (3, 1)`

ÇIKTI

Kod İzleme: `cozum(d)`



x'te en_iyi henüz yok: en_iyi = (3, 1), secim = a.

Adım 3 / 10

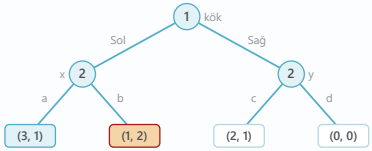
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `v = (3, 1)` `en_iyi = (3, 1)` `secim = a`

ÇIKTI

Kod İzleme: `cozum(d)`



İkinci dal b: (1, 2). Oyuncu 2 kendi kazancına bakar: v[1] = 2 > en_iyi[1] = 1.

Adım 4 / 10

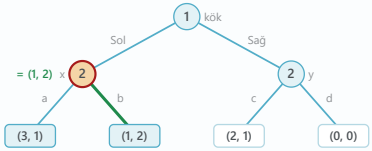
```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

DEĞİŞKENLER

`ad = x` `oyuncu = 1` `hamle = b` `v = (1, 2)` `en_iyi = (3, 1)`

ÇIKTI

Kod İzleme: `cozum(d)`



secim = b; “x b” yazılır, x'in değeri (1, 2). Oyuncu 1'in en sevdiği (3, 1) ulaşılamaz oldu.

Adım 5 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
▶      print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

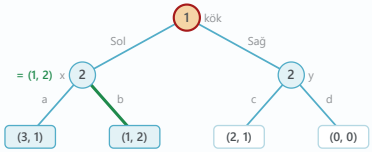
DEĞİŞKENLER

`ad = x` `en_iyi = (1, 2)` `secim = b`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



Kökte Sol'un değeri (1, 2): en_iyi = (1, 2), secim = Sol.

Adım 6 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

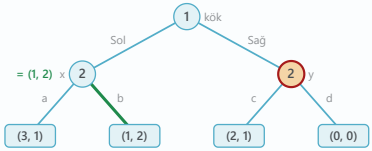
DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `hamle = Sol` `v = (1, 2)` `en_iyi = (1, 2)` `secim = Sol`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



Sağ dalı: cozum(y). c → (2, 1), d → (0, 0); oyuncu 2 için 1 > 0, secim = c.

Adım 7 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
▶      for hamle, cocuk in dallar:
▶          v = cozum(cocuk)
▶          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
▶              en_iyi, secim = v, hamle
10     print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

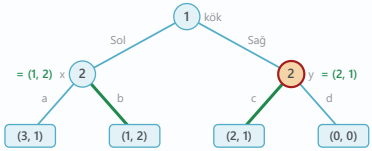
DEĞİŞKENLER

`ad = y` `oyuncu = 1` `en_iyi = (2, 1)` `secim = c`

ÇIKTI

```
x b
```

Kod İzleme: `cozum(d)`



“y c” yazılır; y'nin değeri (2, 1) döner.

Adım 8 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
▶      print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

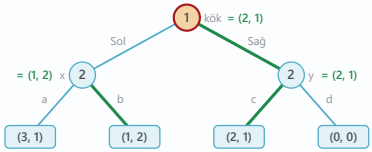
DEĞİŞKENLER

`ad = y` `en_iyi = (2, 1)` `secim = c`

ÇIKTI

```
x b
y c
```

Kod İzleme: `cozum(d)`



Kökte oyuncu 1 karşılaştırır: Sağ 2 > Sol 1 → secim = Sağ. “kök Sağ” yazılır.

Adım 9 / 10

```
 1 def cozum(d):
 2     if isinstance(d, tuple):          # yaprak: kazanç
 3         return d
 4     ad, oyuncu, dallar = d
 5     en_iyi = None
 6     for hamle, cocuk in dallar:
 7         v = cozum(cocuk)
 ▶           if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
 ▶               en_iyi, secim = v, hamle
 ▶       print(ad, secim)
11     return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                  ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
14 print(cozum(kok))
```

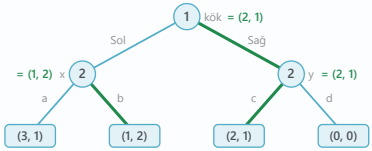
DEĞİŞKENLER

`ad = kök` `oyuncu = 0` `v = (2, 1)` `en_iyi = (2, 1)` `secim = Sağ`

ÇIKTI

```
x b
y c
kök Sağ
```

Kod İzleme: `cozum(d)`



Sonuç (2, 1). SPE: (Sağ; x'te b, y'de c). Ulaşılmayan x'teki b de stratejinin parçasıdır.

Adım 10 / 10

```
1  def cozum(d):
2      if isinstance(d, tuple):          # yaprak: kazanç
3          return d
4      ad, oyuncu, dallar = d
5      en_iyi = None
6      for hamle, cocuk in dallar:
7          v = cozum(cocuk)
8          if en_iyi is None or v[oyuncu] > en_iyi[oyuncu]:
9              en_iyi, secim = v, hamle
10     print(ad, secim)
▶      return en_iyi
12 kok = ['kök', 0, [('Sol', ['x', 1, [('a', (3, 1)), ('b', (1, 2))]]),
13                   ('Sağ', ['y', 1, [('c', (2, 1)), ('d', (0, 0))]])]]
▶ print(cozum(kok))
```

DEĞİŞKENLER

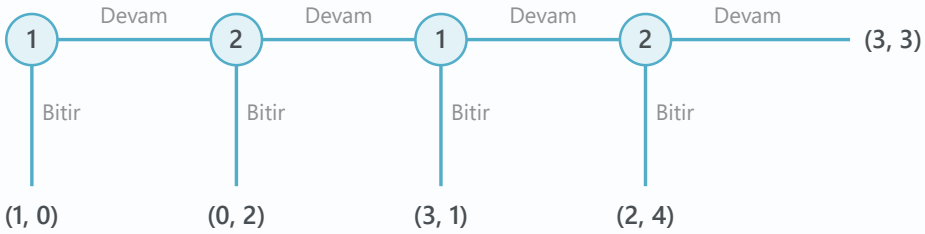
`sonuç = (2, 1)`

ÇIKTI

```
x b
y c
kök Sağ
(2, 1)
```

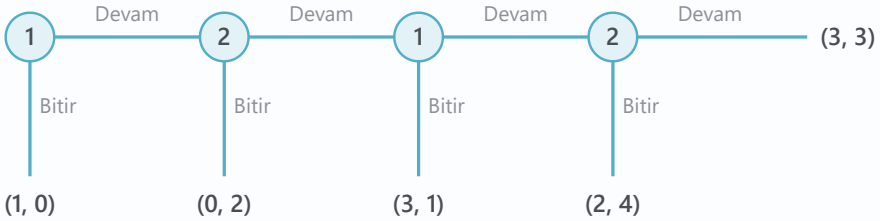
Kırkayak Oyunu (Centipede Game)

- İki oyuncu sırayla karar verir (Rosenthal, 1981). Her düğümde oyuncu ya **Bitir** der ve o anki paylaşım gerçekleşir, ya da **Devam** der ve toplam büyür.
- Toplam kazanç büyür (1 → 2 → 4 → 6; sonda da 6), ama devam eden oyuncu, rakibi bir sonraki adımda bitirirse daha az alır.



Düğümdeki sayı karar veren oyuncudur. Yapraklarda (Oyuncu 1, Oyuncu 2) kazancı; en sağda herkes devam ederse (3, 3).

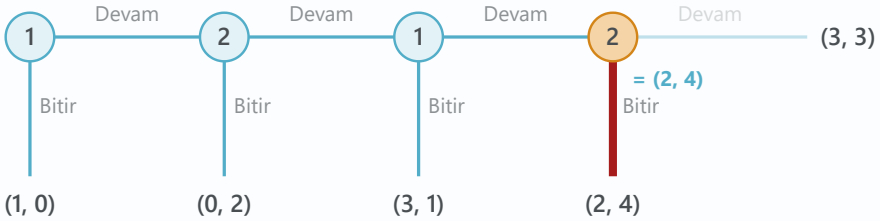
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	?	?	?
3	Oyuncu 1	(3, 1)	?	?	?
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

Oyuncu 1 başlar; her düğümde oyuncu **Bitir** (aşağı) ya da **Devam** (sağa) der. Son karar düğümünden (4. düğüm, Oyuncu 2) başlayıp geriye gideceğiz.

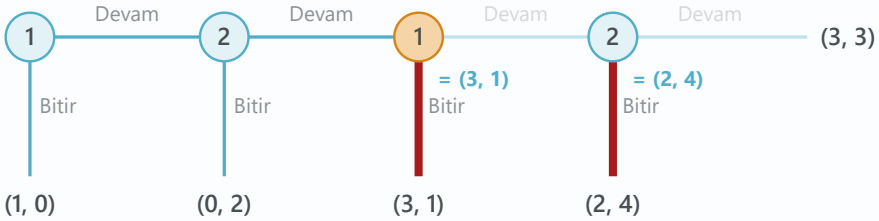
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	?	?	?
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

4. düğüm, Oyuncu 2: Bitir → 4, Devam → 3 (devamın değeri (3, 3)). 4 > 3 ⇒ **Bitir**; düğümün değeri (2, 4).

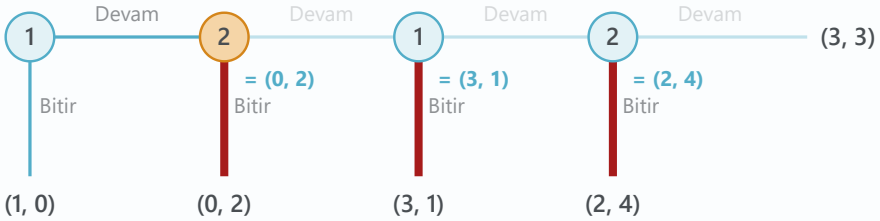
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	?	?	?
1	Oyuncu 1	(1, 0)	?	?	?

3. düğüm, Oyuncu 1: Bitir → 3, Devam → 2 (devamın değeri (2, 4)). 3 > 2 ⇒ **Bitir**; düğümün değeri (3, 1).

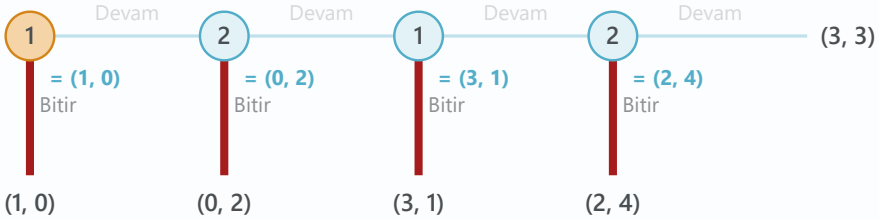
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	?	?	?

2. düğüm, Oyuncu 2: Bitir → 2, Devam → 1 (devamın değeri (3, 1)). 2 > 1 ⇒ **Bitir**; düğümün değeri (0, 2).

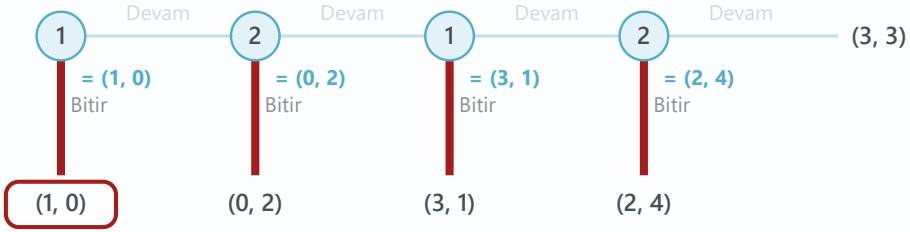
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	(0, 2)	**Bitir**	(1, 0)

1. düğüm, Oyuncu 1: Bitir → 1, Devam → 0 (devamın değeri (0, 2)). 1 > 0 ⇒ **Bitir**; düğümün değeri (1, 0).

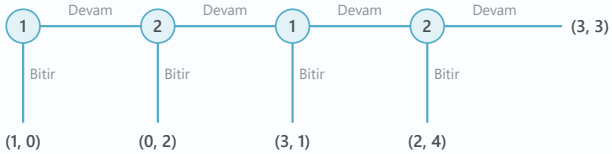
Kırkayak: Geriye Doğru Çıkarım Adım Adım



Düğüm	Karar veren	Bitir	Devamın değeri	Karar	Düğüm değeri
4	Oyuncu 2	(2, 4)	(3, 3)	**Bitir**	(2, 4)
3	Oyuncu 1	(3, 1)	(2, 4)	**Bitir**	(3, 1)
2	Oyuncu 2	(0, 2)	(3, 1)	**Bitir**	(0, 2)
1	Oyuncu 1	(1, 0)	(0, 2)	**Bitir**	(1, 0)

Alt oyun mükemmel denge: her oyuncu **her düğümde Bitir**. Oyun ilk hamlede (1, 0) ile biter; oysa (3, 3) ikisi için de daha iyiydi.

Kod İzleme: `geri_cikarim(dugumler, son)`



deger, oyunun sonundaki kazançla başlar: (3, 3).

Adım 1 / 14

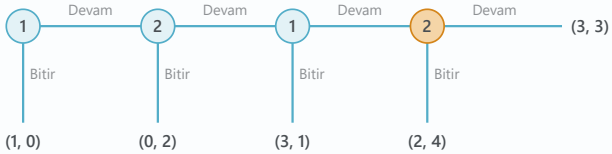
```
1  def geri_cikarim(dugumler, son):
▶    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 3 → 4. düğüm. Karar veren oyuncu indeksi 1 (Oyuncu 2); bitirirse (2, 4).

Adım 2 / 14

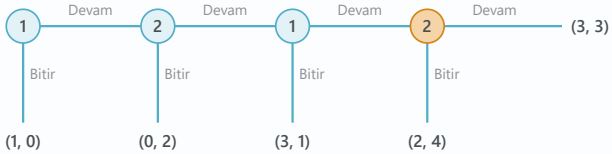
```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[1] = 4 ≥ deger[1] = 3 ? → **Doğru**

Adım 3 / 14

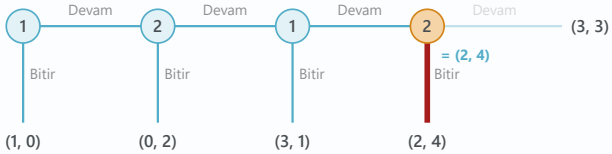
```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (3, 3)`

ÇIKTI

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (2, 4) (4. düğümün değeri).

Adım 4 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

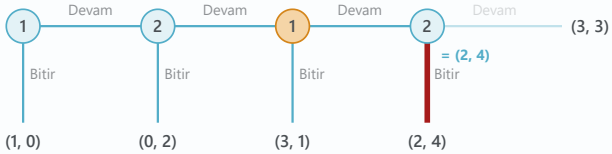
DEĞİŞKENLER

`k = 3` `oyuncu = 1` `bitir = (2, 4)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 2 → 3. düğüm. Karar veren oyuncu indeksi 0 (Oyuncu 1); bitirirse (3, 1).

Adım 5 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

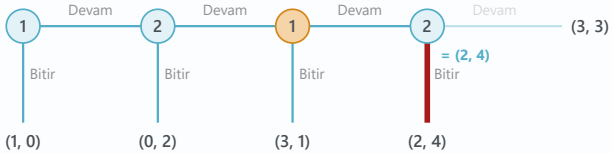
DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[0] = 3 ≥ deger[0] = 2 ? → **Doğru**

Adım 6 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (2, 4)`

ÇIKTI

```
4 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (3, 1) (3. düğümün değeri).

Adım 7 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 2` `oyuncu = 0` `bitir = (3, 1)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 1 → 2. düğüm. Karar veren oyuncu indeksi 1 (Oyuncu 2); bitirirse (0, 2).

Adım 8 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[1] = 2 ≥ deger[1] = 1 ? → **Doğru**

Adım 9 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (3, 1)`

ÇIKTI

```
4 Bitir
3 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (0, 2) (2. düğümün değeri).

Adım 10 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 1` `oyuncu = 1`

`bitir = (0, 2)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



k = 0 → 1. düğüm. Karar veren oyuncu indeksi 0 (Oyuncu 1); bitirirse (1, 0).

Adım 11 / 14

```
1  def geri_cikarim(dugumler, son):
2      deger = son  # oyunun sonu
▶      for k in reversed(range(len(dugumler))):
▶          oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5          if bitir[oyuncu] >= deger[oyuncu]:
6              deger = bitir
7              print(k + 1, "Bitir")
8          else:
9              print(k + 1, "Devam")
10     return deger
```

DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Koşul: bitir[0] = 1 ≥ deger[0] = 0 ? → **Doğru**

Adım 12 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
▶      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (0, 2)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Bitirmek en az devam kadar iyi: deger = (1, 0) (1. düğümün değeri).

Adım 13 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
▶        deger = bitir
▶        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
10   return deger
```

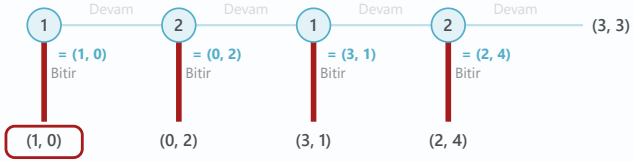
DEĞİŞKENLER

`k = 0` `oyuncu = 0` `bitir = (1, 0)` `deger = (1, 0)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
1 Bitir
```

Kod İzleme: `geri_cikarim(dugumler, son)`



Döngü köke ulaştı: fonksiyon (1, 0) döndürür — alt oyun mükemmel dengenin sonucu.

Adım 14 / 14

```
1  def geri_cikarim(dugumler, son):
2    deger = son  # oyunun sonu
3    for k in reversed(range(len(dugumler))):
4      oyuncu, bitir = dugumler[k]  # 0: O1, 1: O2
5      if bitir[oyuncu] >= deger[oyuncu]:
6        deger = bitir
7        print(k + 1, "Bitir")
8      else:
9        print(k + 1, "Devam")
▶    return deger
```

DEĞİŞKENLER

`deger = (1, 0)` `sonuç = (1, 0)`

ÇIKTI

```
4 Bitir
3 Bitir
2 Bitir
1 Bitir
```

Kırkayak Oyunu: Teori ve Deney

Teori (SPE)

Her oyuncu her düğümde Bitir der; oyun ilk hamlede (1, 0) ile biter. Oyunun herhangi bir Nash dengesinde de sonuç aynıdır.

Verimsizlik

(3, 3) her iki oyuncu için de (1, 0)'dan iyidir: SPE sonucu **Pareto optimal değildir**. Düğüm sayısı arttıkça kayıp büyür.

Deney

Laboratuvarda oyuncuların çoğu ilk düğümde bitirmez; birkaç tur devam eder (McKelvey ve Palfrey, 1992).

Neden?

Geriye doğru çıkarım, rasyonelliğin **ortak bilgi** olmasını ve uzun bir akıl yürütme zincirini gerektirir; rakibin biraz "iyi niyetli" olma ihtimali bile devam etmeyi kârlı kılar.

Kırkayak, sonlu tekrarlanan Mahkûmun İkilemi ile aynı mantığı paylaşır: son turdan geriye bakınca işbirliği çözülür. Sonsuz ufukta ise işbirliği dengede kalabilir.

ÖZET

Altı Klasik Oyunun Karşılaştırması

Oyun	Sıfır toplamlı	Simetrik	Saf NE	Karışık NE (satır, sütun)	(u_1, u_2)
Yazı-Tura Eşleştirme	evet	hayır	0	p* = 1/2, q* = 1/2	(0, 0)
Tavuk	hayır	evet	2	p* = 9/10, q* = 9/10	(−1/10, −1/10)
Cinsiyetler Savaşı	hayır	hayır	2	p* = 3/5, q* = 2/5	(6/5, 6/5)
Geyik Avı	hayır	evet	2	p* = 1/2, q* = 1/2	(1, 1)
Saf Koordinasyon	hayır	evet	2	p* = 1/3, q* = 1/3	(10/3, 10/3)
Bedava Para	hayır	evet	2	yok (yalnız uç)	—

p* = satır oyuncusunun ilk stratejisinin (tablodaki ilk satır), q* = sütun oyuncusunun ilk stratejisinin olasılığı. Son sütun karışık dengedeki beklenen kazançlar. Mahkûmun İkilemi: 1 saf NE (baskın stratejilerle), karışık NE yok.

ÖZET

Özet

Temsil

Normal form (matris, eşzamanlı) ve genişletilmiş form (ağaç, sıralı).

Sınıflandırma

Zamanlama, işbirliği, toplam kazanç, simetri, bilgi (mükemmel / tam) ve ufuk birbirinden bağımsız boyutlardır.

Saf denge

En iyi tepkileri işaretle; iki işaretli hücre saf Nash dengesidir. Yazı-Tura'da yok, koordinasyon oyunlarında iki tane.

Karışık denge

Her oyuncu rakibini kayıtsız bırakan olasılığı oynar; sonuç çoğu zaman verimsizdir (Cinsiyetler Savaşı: %52 buluşamama).

Denge seçimi

Çoklu denge varsa Pareto, risk, odak noktası ve zayıf/kesin denge ayrımı hangisinin oynanacağını etkiler.

Geriye doğru çıkarım

Sıralı oyunları yapraktan köke çözer, alt oyun mükemmel dengeyi verir ve inandırıcı olmayan tehditleri eler.

ÖZET

Dersin Devamında

Nash dengesi
Biçimsel tanım, varlık, minimaks, karmaşıklık.
Bölüm 3

Mekanizma tasarımı
VCG, teşvik uyumluluğu, Bayesçi oyunlar.
Bölüm 4

Açık artırmalar
İngiliz, Hollanda, birinci ve ikinci fiyat.
Bölüm 5

Sosyal seçim
Oylama kuralları, Arrow teoremi.
Bölüm 6

Koalisyon oyunları
Shapley değeri, çekirdek.
Bölüm 7

Tekrarlanan oyunlar
Öğrenme, Tit-for-Tat.
Bölüm 12

ÖZET

Etkileşimli Oyunlar

Bu bölümün oyunları iki kişilik, sıralı, **mükemmel bilgili**, sonlu ve (kazan/kaybet açısından) sıfır toplamlıdır: kuramsal olarak geriye doğru çıkarımla çözülebilirler, ama ağaç çok büyüktür.

Hex

Beraberlik yoktur; strateji çalma argümanıyla ilk oyuncunun kazanan stratejisi olduğu kanıtlanır.

Connect Four

1988'de çözüldü: mükemmel oyunda ilk oyuncu kazanır.

Othello (Reversi)

8×8 tahtada mükemmel oyunun beraberlikle bittiği 2023'te hesaplamayla gösterildi. Taş sayısından çok hareketlilik (yasal hamle sayısı) önemlidir.

Tahta büyüdükçe bu oyunlar hesaplama açısından zorlaşır: genelleştirilmiş n×n Hex PSPACE-tamdır (Reisch, 1981). Bu tür sonuçların derlemesi: Hearn ve Demaine, *Games, Puzzles, and Computation*.

ÖZET

Örnek Problemler

1. (A, A) = (4, 1), (B, B) = (1, 4), diğer hücreler (0, 0). Saf ve karışık dengeler? Buluşamama olasılığı?
2. Tavuk'ta çarpışma −20 olsaydı karışık dengede kırma ve çarpışma olasılıkları?
3. Geyik Avı'nda tavşan 1,5 kazandırsaydı karışık dengede geyik olasılığı? Güven kolaylaşır mı?
4. Altı oyundan hangileri sıfır toplamlı, hangileri simetrik?
5. Kırkayak'ta en sağdaki kazanç (5, 5) olsaydı alt oyun mükemmel denge?
6. Piyasaya girişte (Girme, Savaş) neden alt oyun mükemmel değil?

▶ **Cevapları göster**

Kaynaklar

- M. J. Osborne, *An Introduction to Game Theory*, Oxford University Press, 2004 (Bölüm 2–5).
- N. Nisan, T. Roughgarden, É. Tardos, V. Vazirani (ed.), *Algorithmic Game Theory*, Cambridge University Press, 2007 (Bölüm 1).
- Y. Shoham, K. Leyton-Brown, *Multiagent Systems*, Cambridge University Press, 2009 (Bölüm 3 ve 5).
- R. W. Rosenthal, "Games of perfect information, predatory pricing and the chain-store paradox", *J. Economic Theory* 25, 1981.
- R. D. McKelvey, T. R. Palfrey, "An experimental study of the centipede game", *Econometrica* 60(4), 1992.
- R. Wilson, "Computing equilibria of n-person games", *SIAM J. Applied Mathematics* 21, 1971.
- B. Skyrms, *The Stag Hunt and the Evolution of Social Structure*, Cambridge University Press, 2004.
- A. R. Karlin, Y. Peres, *Game Theory, Alive*, American Mathematical Society, 2017 (2×2 oyunlar, karışık denge).
- R. A. Hearn, E. D. Demaine, *Games, Puzzles, and Computation*, A K Peters, 2009 (Hex gibi oyunların karmaşıklığı).
- S. Reisch, "Hex ist PSPACE-vollständig", *Acta Informatica* 15, 1981.
- Ders sayfası: **T. Roughgarden, CS364A Algorithmic Game Theory** (Stanford).
- Araçlar: **Gambit** (oyun ağaçları ve matris oyunlarında denge hesabı) · **Nashpy** (iki oyunculu oyunlar için Python kütüphanesi; karisik_2x2 sonuçlarını denetlemek için).

Son

Bölüm 2: Oyun Kuramına Giriş

Sonraki bölüm: Bölüm 3: Nash Dengesi

Çalışma soruları · Terim sözlüğü